Introduction to Software Project Management

Unit_1

Introduction

- “Most software projects fail due to mismanagement, not just bad code. This course will teach you how to plan, manage, and succeed in real-world software development.”
- “More than 60% of software projects run over time or budget. Why? Let’s find out.”

Software Project Management (SPM)

- SPM is the discipline of planning, organizing, and managing software projects.
- Focuses on delivering software products on time, within budget, and meeting quality.
- Involves resource management, risk management, cost estimation, and change control.
- Ensures alignment between technical execution and business goals.

Historical Background and Context

- 1960s: Software crisis revealed project failures in cost, schedule, quality.
- 1970s–80s: Waterfall model introduced for structure; rigid sequential phases.
- 1990s: PMBOK and PRINCE2 formalized project management knowledge areas.
- 2000s–present: Agile and iterative methods introduced to improve flexibility and user involvement.

Software Crisis?

- The **“software crisis”** refers to a period during the **late 1960s through the 1970s** when large-scale software projects increasingly **failed to meet expectations** — they were **over budget, behind schedule, and often didn’t work**.
- This crisis led to a turning point in how software development was perceived and managed.

Why Did the Crisis Occur?

- **Exploding Demand for Software**
  - The rise of mainframe computers in government, military, and business meant software was needed on **a much larger scale**.
  - Projects became **too complex** to be handled by informal or ad-hoc coding.
- **Poor Project Planning and Estimation**
  - Projects lacked structured management.
  - Delivery deadlines and budgets were often **wildly inaccurate**.
- **Changing or Unclear Requirements**
  - Clients often didn't know exactly what they wanted.
  - Developers built the wrong product — or the right product too late.
- **No Formal Processes**
  - There were **no standardized methods** for testing, requirements gathering, or version control.
  - Coding was seen as a "craft" rather than an engineering discipline.
- **Lack of Tools and Team Coordination**
  - No IDEs, no Git, no Agile — developers often worked in isolation.
  - Communication breakdowns were common in large teams.

Notable Project Failures

- **IBM OS/360**
  - One of the earliest and largest software projects. It was years late and over budget by hundreds of millions of dollars. But it also sparked many innovations (e.g., modular programming, testing practices).
- **US Air Defense System (SAGE)**
  - Though pioneering, it highlighted the difficulty of managing **real-time, mission-critical systems** with thousands of lines of code.

Resulting Reactions

- **Birth of Software Engineering (1968)**
  - The term *"software engineering"* was coined at a NATO conference in Germany.
  - The goal: apply engineering discipline to software development.
- **Introduction of Models and Frameworks**
  - Waterfall Model (Winston Royce, 1970): introduced phased development.
  - Structured programming and modularization began to emerge.
- **Emergence of Software Project Management**
  - Project management became **a distinct discipline** in software development.
  - Tools like **PERT charts, Gantt charts**, and estimation techniques were adapted from civil engineering.

Impact on Modern SPM

- The crisis revealed that **coding alone is not enough** — software needs to be planned, budgeted, tested, and reviewed like any other product.
- It pushed the industry toward:
  - **Standard methodologies** (PMBOK, PRINCE2)
  - **Disciplined testing and versioning**
  - **Project management roles** and training
  - **Modern practices** like Agile and DevOps to respond to change more rapidly

Need and Importance of SPM

- Software development is complex, evolving, and often unpredictable.
- SPM provides structure, planning, and visibility into project status.
- Minimizes risks through early identification and mitigation strategies.
- Ensures stakeholder alignment, user satisfaction, and product quality.

Advantages of Software Project Management

- Improved planning, scheduling, and resource allocation.
- Better communication among stakeholders and team members.
- Enhanced risk management and quality assurance processes.
- Greater transparency, traceability, and project control.

Disadvantages and Challenges

- May introduce overhead with excessive documentation and process.
- Rigid management may reduce agility and innovation.
- Requires skilled managers and leadership commitment.
- Not a guarantee of success—relies on proper execution and team collaboration.

🏠 Building a House vs 📱 Developing an App

Aspect	Building a House	Developing a Mobile App
Product Nature	Tangible, visible physical structure	Intangible, code-based software
Design Finalization	Must be finalized before construction begins	Can be revised during development (flexible scope)
Change Flexibility	Changes after construction starts are costly	Can implement changes even late in development
Stakeholder Involvement	Involved mostly in early and final stages	Involved throughout (sprints, feedback loops, testing)

🏠 Building a House vs 📱 Developing an App

Aspect	Building a House	Developing a Mobile App
Testing & Quality Check	Inspections done at milestones (foundation, wiring)	Continuous testing (unit, integration, UAT, beta testing)
Progress Visibility	Visibly trackable (walls, windows, paint, etc.)	Invisible progress (tracked via features, tests, demos)
Dependencies	Materials, labor, weather, land, regulations	APIs, platforms, users, device compatibility
Methodology	Waterfall (sequential: plan → build → deliver)	Agile/Iterative (develop, test, review, repeat)

🏠 Building a House vs 📱 Developing an App

Aspect	Building a House	Developing a Mobile App
Methodology	Waterfall (sequential: plan → build → deliver)	Agile/Iterative (develop, test, review, repeat)
Failure Impact	Failure is often visible and costly to reverse	Failures may be silent (bugs, performance issues)
Resource Types	Bricks, cement, plumbers, electricians	Developers, designers, testers, cloud platforms
Documentation Importance	Blueprints essential and legally binding	Documentation optional or lightweight in Agile
Post-Delivery Changes	Major renovations needed (costly, time-consuming)	Can push app updates anytime (via app store or servers)

Software Project Management and General Project Management

Aspect	General Project Management (e.g., Construction, Manufacturing)	Software Project Management (Software Development)
Nature of Product	Tangible, physical product (building, device). Progress is visible as concrete stages completed.	Intangible product (software code and functionality). Progress is less visible, often tracked via working features or milestones.
Requirements Stability	Typically more stable after initial planning; changes are difficult and costly once execution starts.	Requirements are prone to change; project often accommodates evolving needs and refinements throughout its life cycle. Agile processes embrace change.

Software Project Management and General Project Management

Aspect	General Project Management (e.g., Construction, Manufacturing)	Software Project Management (Software Development)
Flexibility to Change	Low flexibility – constrained by physical laws and irreversible decisions (can't easily "un-build" part of a bridge). Revisions can cause major delays/costs.	High flexibility – design is malleable. Can often revise or pivot the software even late in development at relatively lower cost. Iteration and course-correction are common.
Resource Focus	Mix of human resources and physical resources (materials, equipment). Management includes supply chain and logistics for physical items.	Primarily human resources (developers, designers, testers). Management focuses on team skills, communication, and productivity. The work is people-intensive with minimal physical material concerns.

Software Project Management and General Project Management

Aspect	General Project Management (e.g., Construction, Manufacturing)	Software Project Management (Software Development)
Approach & Lifecycle	Often linear and milestone-based (e.g., design → build → deliver). Uses traditional PM tools (Gantt charts, critical path) with well-defined phases.	Often iterative/incremental (e.g., multiple development sprints). Uses both traditional PM tools and software-specific frameworks (Agile/Scrum boards, continuous integration). Development and management activities may overlap in cycles.
Quality Assurance	Quality checks at specific milestones (inspections, final testing of product functionality or safety). Defects often found late are hard to fix without rework.	Continuous quality assurance (ongoing testing, code reviews throughout development). Bugs can be fixed iteratively. Software quality includes metrics like reliability, performance

Software Project Management and General Project Management

Aspect	General Project Management (e.g., Construction, Manufacturing)	Software Project Management (Software Development)
Stakeholder Involvement	Stakeholders (clients, sponsors) involved mainly at start (requirements/design approval) and end (acceptance), with periodic status reports. External parties (suppliers, regulators) play roles at specific points.	Stakeholders (clients, end-users) ideally engaged throughout. Frequent demos, feedback cycles, and updates are provided to ensure the software meets expectations. Fewer external dependencies beyond the client, enabling focus on the customer-team feedback loop.

Brainstorming:

- “Why do you think traditional project management doesn’t work well for software?”

Key Roles in Software Project Management

- Project Sponsor: Provides funding and strategic direction.
- Project Manager: Oversees planning, execution, and control.
- Product Owner/BA: Defines requirements and prioritizes features.
- Software Developers: Build and maintain software functionality.

Additional Roles in SPM

- QA/Testers: Ensure software quality through testing strategies.
- UI/UX Designers: Improve usability and user satisfaction.
- Scrum Master: Facilitates Agile ceremonies and removes blockers.
- Stakeholders: Provide feedback, validate deliverables, and accept final product.

Activity idea:

- Form groups and assign each group a role. Ask them to describe what that role contributes to a software project.

Success Example: Nagarik App (Government of Nepal)

- **Project Description:** Nagarik App is a digital platform launched by the Government of Nepal to provide citizens with access to various public services (citizenship details, PAN registration, educational certificates, etc.) through a single mobile app.
- **Relevance to SPM:**
- **Clear goals and vision** defined by the Ministry of Communication and Information Technology.
- Project followed an **iterative development model**, launching in phases with increasing features.
- Involved **multiple stakeholders**: NITC, private software firms, NEA, MoHA, etc.
- **Challenges managed**: user data integration, privacy concerns, and testing on various devices.
- **SPM Takeaways:**
- Strong coordination among government bodies and vendors.
- Scope was phased to ensure initial delivery and later enhancements.
- Regular public feedback was incorporated—showing **Agile principles** in public sector.

Failure/Risk Example: National ID Management Project

- **Project Description:** The National ID card initiative aimed to implement biometric-based identification for all citizens. However, the rollout has been **delayed by several years**.
- **Relevance to SPM:**
- Poor **stakeholder coordination** (MoHA, Department of National ID, local governments).
- Lack of **technical capacity planning** and underestimated integration issues.
- Issues with **procurement delays**, vendor management, and infrastructure.
- **SPM Learnings:**
- Weak risk management and over-ambitious timelines led to missed deadlines.
- **Inadequate communication** between software team and field-level implementers.
- Project scope and requirement analysis were not clear during planning.

Private Sector Success: eSewa Mobile Wallet

- **Project Description:** eSewa, Nepal's first digital wallet, successfully scaled into a national payments platform offering utility payments, remittance, airline booking, and more.
- **SPM Highlights:**
- **Incremental feature release**: launched core wallet first, then added services.
- **User-centric design** led by continuous testing and feedback.
- Strong **project leadership** and resource allocation helped it scale reliably.
- **SPM Application:**
- Used Agile-style iterative development.
- Prioritized **security, performance, and scalability** as key project goals.
- **Stakeholder alignment** (banks, NRB guidelines, users) was consistently maintained.

Online Driving License Application System (DoTM)

- **Context:**
  The Department of Transport Management (DoTM) launched an online license application portal to digitize the process of applying for a driving license.
- **Challenges & SPM Failures:**
- **System crashes frequently** during peak times due to poor load testing.
- **No user feedback loop** — many users struggled with verification and appointment booking.
- **Inadequate stakeholder coordination** with local offices and exam centers.
- **Lack of scalability planning**: the system wasn't prepared for the nationwide load.
- **SPM Lessons:**
- Absence of performance testing and risk mitigation plans.
- No clear communication plan or user support strategy.
- Failure to anticipate user volume growth.

Online Education Portals During COVID-19 (Various Municipalities)

- **Context:**
  During the pandemic, many municipalities launched learning portals for public school students.
- **Challenges & SPM Failures:**
- Many platforms **were not mobile-friendly** — majority of target users accessed via phones.
- **Lack of clear requirements and user research** — no pilot testing before full deployment.
- **Content management was poor**, and most portals were inactive after a few weeks.
- **Low adoption** by teachers and students due to poor usability and lack of training.
- **SPM Lessons:**
- Ignoring end-user needs in requirement analysis leads to low usability.
- Failure to plan for user training and support caused abandonment.
- No stakeholder engagement (e.g., teachers, parents) in the development process.

National eHealth Project (Ministry of Health)

- **Context:**
  A national plan to digitize hospitals and health posts, including EMR (Electronic Medical Records) and patient tracking systems.
- **Challenges & SPM Failures:**
- **Delayed procurement and unclear technical specifications.**
- **Fragmented implementation** — different systems were installed in different hospitals without integration.
- **Vendor mismanagement** and absence of a unified roadmap caused misalignment.
- **Lack of training** and IT support at local levels meant systems weren't used even after delivery.
- **SPM Lessons:**
- No unified requirement specification or architecture plan.
- Poor coordination across central and provincial levels.
- Insufficient planning for capacity building and sustainability.

Election Commission's Real-Time Vote Counting System (Attempted 2017)

- **Context:**
  ECN attempted to implement real-time vote counting dashboards.
- **Challenges & SPM Failures:**
- **System failed under load** during peak result hours.
- Poor **data synchronization** between local vote centers and central servers.
- **Inconsistent user interfaces** led to miscommunication in early results.
- Project not thoroughly tested before real deployment.
- **SPM Lessons:**
- Lack of stress testing and fallback mechanisms.
- Inadequate disaster recovery and data integrity checks.
- Last-minute deployment without stakeholder training.

Unit 2: Project Life Cycle and Processes

BESE 7th Semester

- **“Every project you have seen, from building a house in your village to developing an app in Pokhara, follows a certain path. What do you think happens between the first idea and the final handover?”**

Why the chapter matters

- **How real projects move from start to end**, and why missing a step leads to delay, cost overruns, and confusion.
- Example:

"In Nepal, several municipal e-governance projects failed because teams jumped into coding before defining the scope. Understanding the life cycle prevents the same mistake in your own work."

Project Life Cycle

- **Example: Building a small mobile app for a local trekking agency in Lakeside, Pokhara.**
- You can show that the project moves through:
  - identifying the need (agency wants online booking)
  - planning (what features, timeline, cost)
  - execution (development and testing)
  - closure (handover, training, and support)

Project Life Cycle

- Project life cycle is a series of phases of a project from initiation to completion.
- The life cycle gives a practical approach to problem solving applied to all aspects of a project.
- Phases in a project life cycle encompasses sequential and overlapping phases.
- A project life cycle typically has 4 major phases:
  - Initiation Phase
  - Planning Phase
  - Implementation/Execution Phase
  - Closure Phase

- **Initiation Phase:**
  - The project need is identified in the initiation phase. It signals the start of a project.
- **Planning Phase:**
  - This phase is where the project solution is further explored and the steps necessary to achieve their goals.
- **Implementation Phase:**
  - The project plan is put into action. This phase is when people are carrying out the tasks of their plan.
- **Closure Phase:**
  - This phase is the final closing of all project processes and handing over deliverables to the customer.

- **Initiation Phase**
- **Example:**
- Pokhara Metropolitan decides to build a digital house-map system.
- During initiation, they only identify the need, confirm feasibility, and assign a team.
- No design or coding happens here.

- **Planning Phase**
- **Example:**
- A private bank in Kathmandu wants to introduce mobile loan applications.
- Planning includes timelines, a cost sheet, a resource plan, risk list, and communication plan.
- Show how skipping this step causes issues:
  - Budget underestimation
  - Missed deadlines
  - Confusion among developers and stakeholders

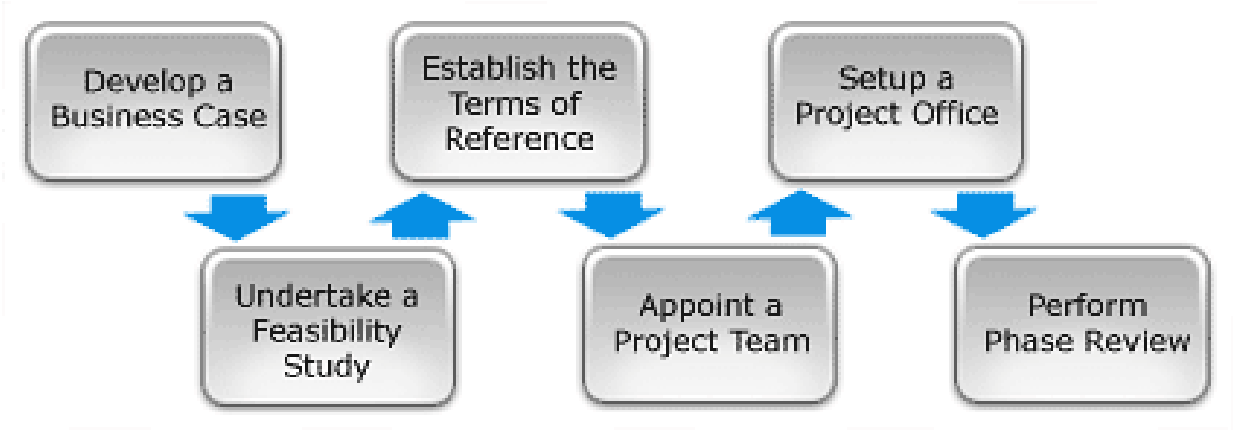
- **Execution Phase**
- **Example:**
- Developing Nepal Airlines' ticketing portal upgrade.
- Execution includes coding, testing, integrating payment gateways, security checks, and content upload.
- This phase consumes the most effort.

- **Closure Phase**
- **Example:**
- A hydropower monitoring system project in Kaski is ready for use.
- Closure includes documentation handover, final meeting, acceptance letter, and training.

Project Life Cycle

Project Initiation Phase

- The **Project Initiation Phase** is the 1st phase in the *Project Life Cycle*, as it involves starting up a new project.
- One can **start a new project by defining its objectives, scope, purpose and deliverables** to be produced.
- One will also **hire project team**, **setup the Project Office** and **review the project**.



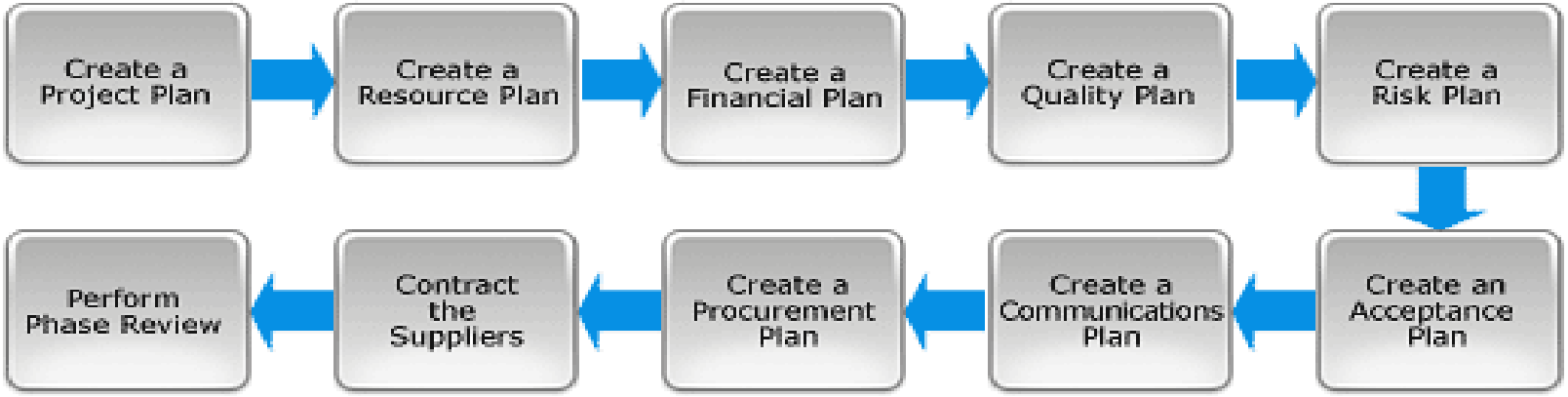
Initiation

TABLE 1.2 Initiation Phase

Phase	*Task Description*	*Output*
Initiation	✔ Broad direction for the project is provided to the project leader by the sponsor. Limits, constraints, and project priorities are defined.	Charter

Project Planning Phase

- The **Project Planning Phase** is the second phase in the *project life cycle*.
- It involves creating of a set of plans to help guide the team through the execution and closure phases of the project.
- The plans created during this phase **will help to manage time, cost, quality, change, risk** and issues.
- They will also help **manage staff and external suppliers**, to ensure that you deliver the project on time and within budget.



Planning

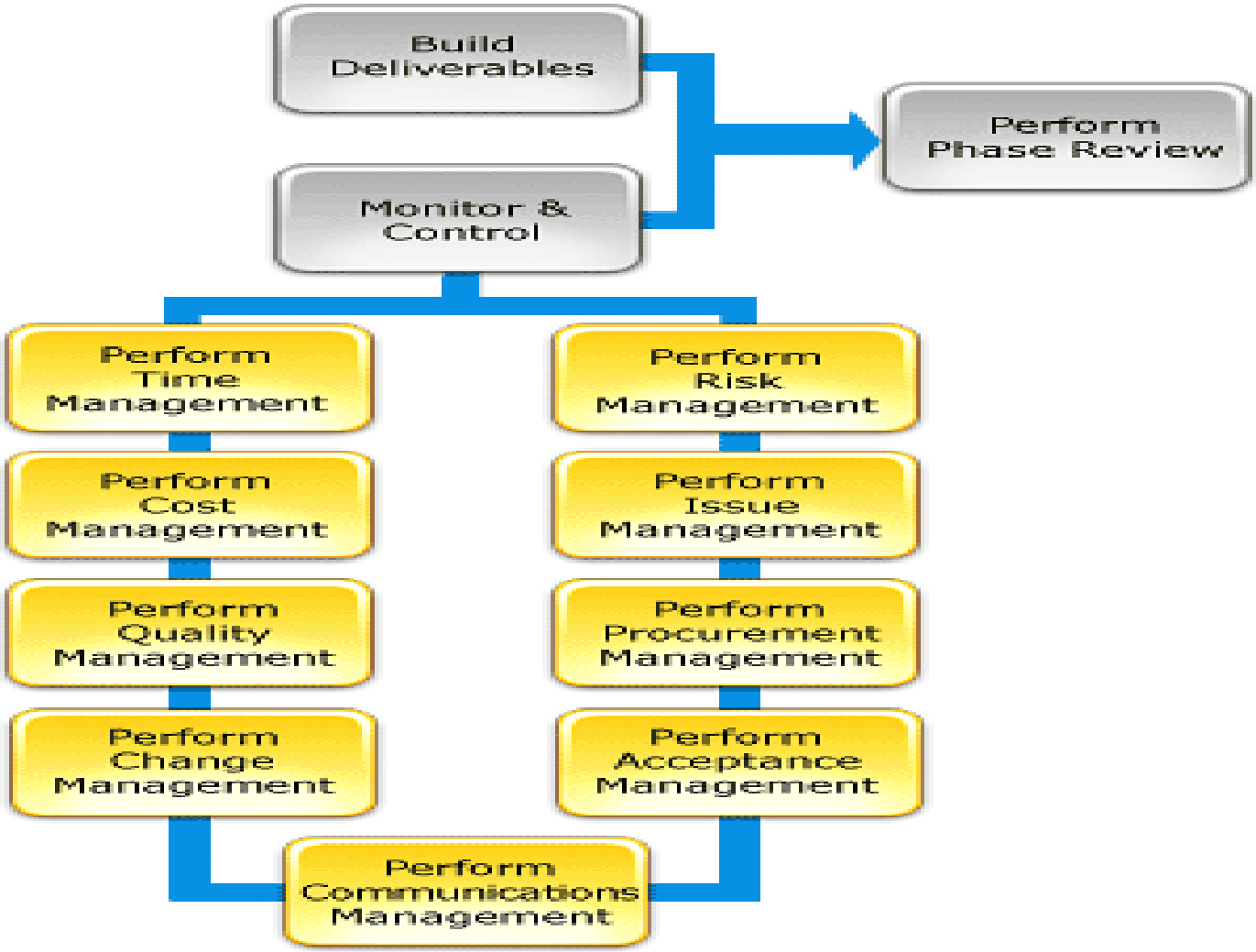
TABLE 1.3 Planning Phase

Phase	*Task Description*	*Output*
Planning	✔ Select the team members ✔ Define the scope of the project ✔ Define any risks associated with the project and develop ways to prevent them ✔ Determine the resources required to complete the project	Project Plan

Project Execution Phase

- The **Project Execution Phase** is the third phase in the *project life cycle*.
- One will **build the physical project deliverables** and **present them to the customer**.
- This is usually the **longest phase** in the project life cycle and it typically **consumes the most energy and the most resources.**

Project Execution Phase

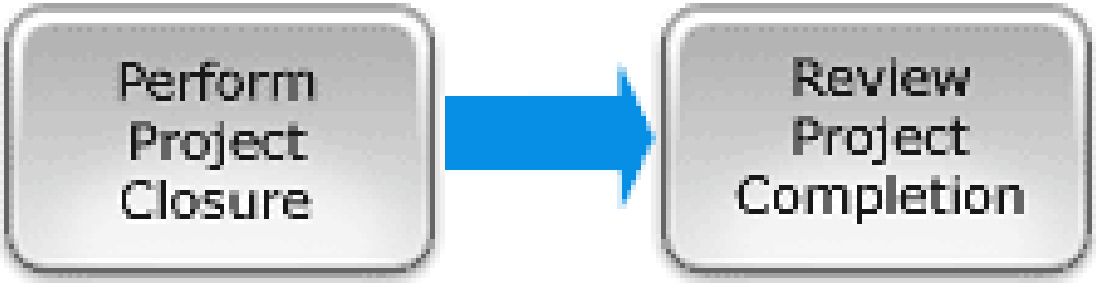


Execution

TABLE 1.4 Execution Phase		
Phase	*Task Description*	*Output*
Execution	✔ Create the deliverables	Status reports
	✔ Monitor project progress	Final deliverable
	✔ Resolve issues	
	✔ Communicate progress	
	✔ Manage changes to the plan	

Project Closure Phase

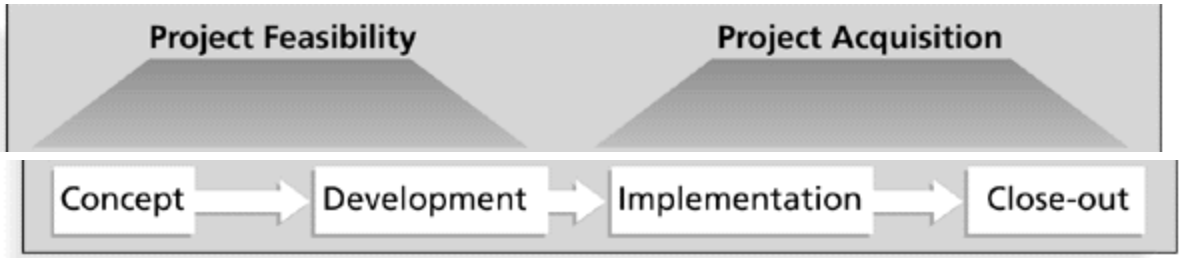
- The **Project Closure Phase** is the fourth and last phase in the *project life cycle*.
- In this phase, one will **formally close the project** and then **report its overall level of success** to the sponsor.
- Project Closure involves **handing over the deliverables to the customer**, **passing the documentation to the business**, **cancelling supplier contracts**, **releasing staff and equipment**, and **informing stakeholders of the closure of the project**.



Closure

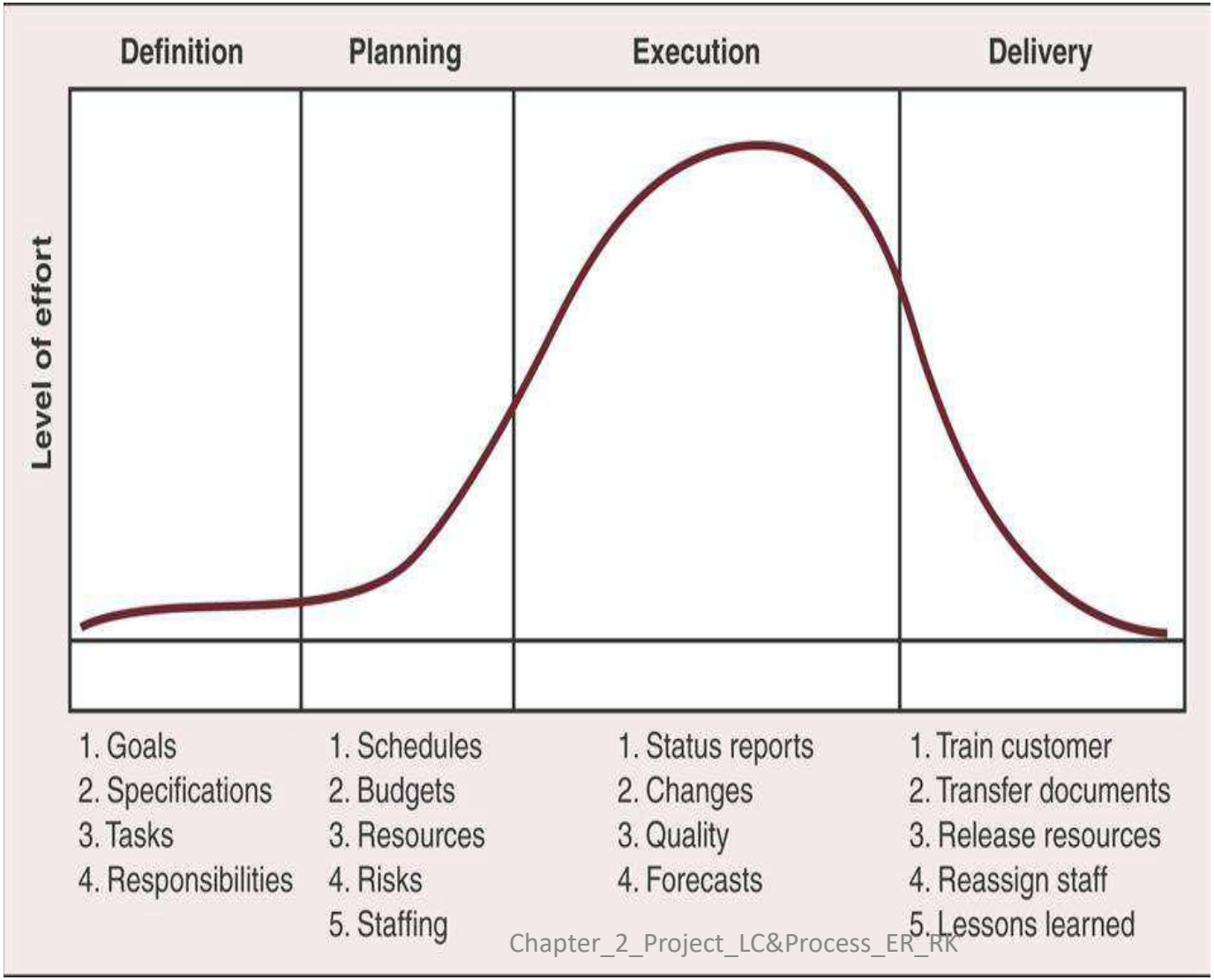
TABLE 1.5 Close-Out Phase

Phase	*Task Description*	*Output*
Close out	✔ Evaluation of customer satisfaction ✔ Assessment of lessons learned	Close-out report



Management plan	Project plan	Last work package	Completed work
Preliminary cost estimate	Budgetary cost estimate	Definitive cost estimate	Lessons learned
3-level WBS	6+-level WBS	Bulk of time should be spent here	Customer acceptance

Project Life Cycle : An Overview



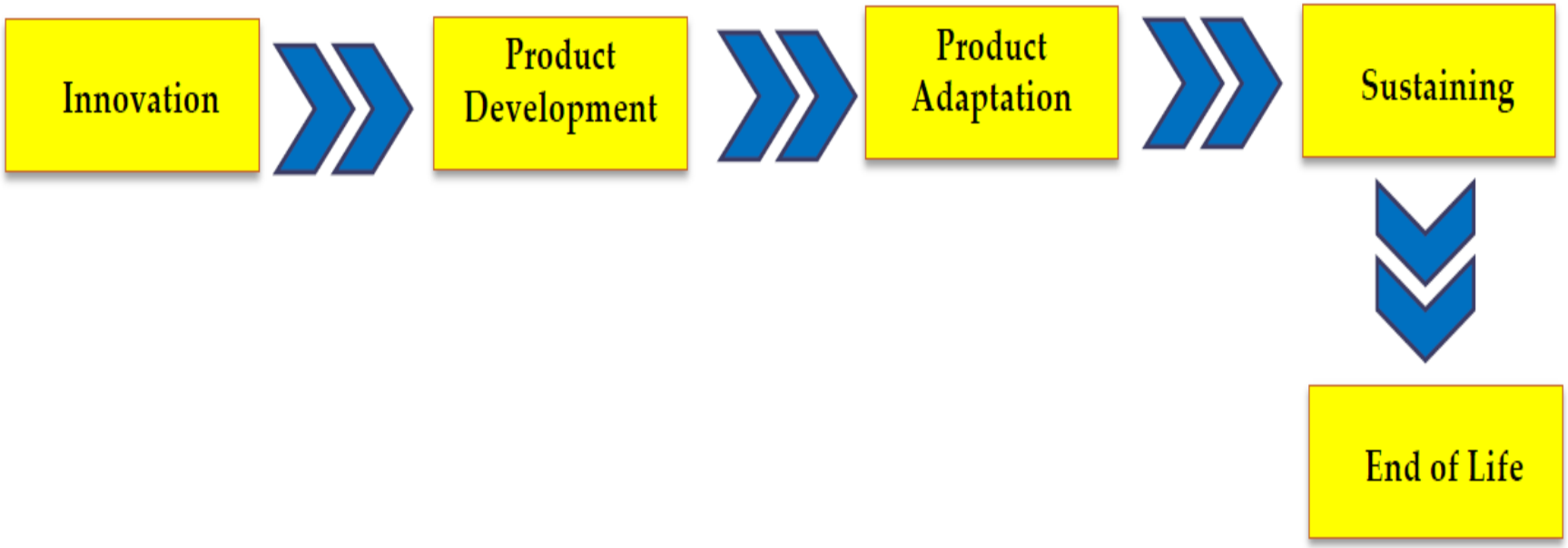
Product Life Cycle

- A **product is anything that can be offered to a market to solve a problem, or to satisfy a want or need.**
- Products have a life cycle that consists of multiple stages. First the product is conceived, then developed, then introduced and managed in the market, and finally the product is retired when the need for it diminishes.
- A **project is a temporary endeavor that is undertaken to create a unique product or service. With a project, there is a clear definition of what needs to be delivered by a specified date in time.**
- It's important to note that a product can only be developed within the context of a project, and multiple projects can occur within a product's life cycle.
- The product life cycle starts with the idea of a new product in an organization that gets evaluated during the innovation phase which is followed by the product development phase.
- After the initial market introduction, the product is adapted and then sustained and finally reaches its end of life

Product Life Cycle

- The product life cycle ***starts with the idea of a new product*** in an organization that gets evaluated during the innovation phase, which is followed by the product development phase.
- After the initial market introduction, the product is adapted and then sustained, and finally reaches its end of life

- *Phases of product life cycle*



Product Life Cycle

Review Milestone	Milestone Questions
End Of Innovation	Does a product idea exist that is ready for production? Is the available product idea good for the company and its target market?
End of Product Development	Is the implementation done correctly? Is the product functionality meeting customer needs? Is the software product ready for delivery? Is marketing ready to launch the product?
End of product Adaptation	Do existing product modifications meet customer demands?
End of Sustaining	How big is the existing customer base that is actively using this product? Is this customer base ready to transfer to a new product? Can resources supporting this product be shifted to other product developments?
End of life	Have all customers been transferred to the new product? Has the infrastructure, the product and its documentation been archived?

Stepwise Project Planning Process

Step 0: Select project

Step 1: Identify project scope and objectives

Step 2: Identify project infrastructure

Step 3: Analyze project characteristics

Step 4: Identify project products and activities

Step 5: Estimate effort for each activity

Step 6: Identify activity risks

Step 7: Allocate resources

Step 8: Review/publicize plan

Step 9: Execute plan

Step 10: Execute lower levels of planning

Project Planning

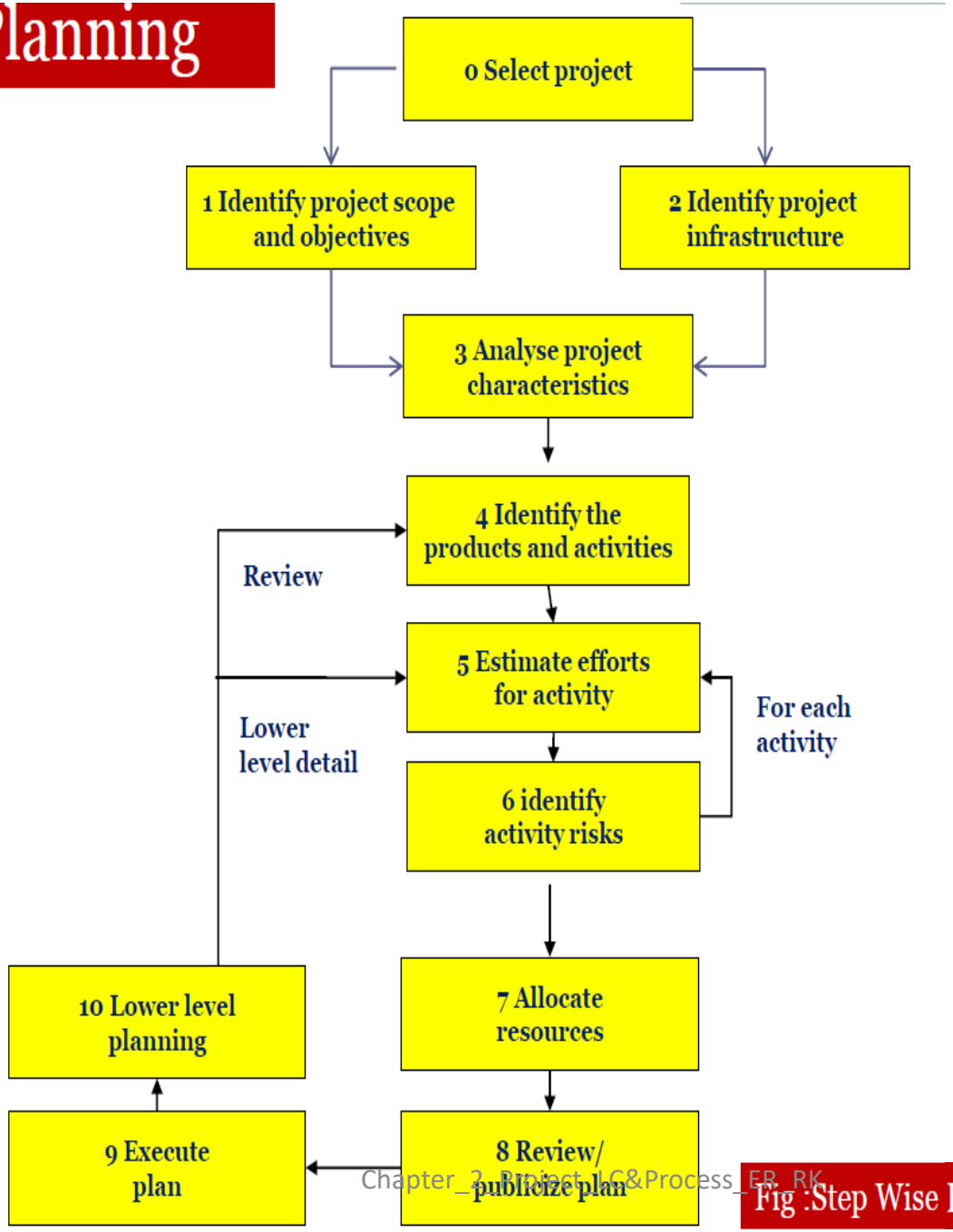


Fig :Step Wise Planning Activity

Step 0: Select Project

- This is called step 0 because in a way of project planning, it is outside the main project planning process.
- Feasibility study suggests us that the project is worthwhile or not.

Step 1: Identify project scope and objectives

- Identify objectives and measures of the effectiveness in meeting those objectives
- Establish a project authority.
  - To ensure the unity of purpose among all persons concerned.
- Identify all stakeholders in the project and their interests
- Modify objectives in the light of stakeholder analysis if necessary
- Establish methods of communication between all parties

Step 2:Identify Project Infrastructure

- **Identify relationship between the project and strategic planning**
  - To determine the order of related projects (in the organization) being carried out.
  - To establish a framework within which the system fits.
  - To ensure the hardware and software standards are followed.
- **Identify installation standards and procedures.**
  - More appropriate name: “Identify standards and procedures related to the software project.”
- **Identify project team organization**

Step 3: Analyze Project Characteristics

- Distinguish the project as either objective-driven or product driven.
- Analyze other project characteristics (including quality-based ones)
- Identify high level project risks
- Take into account user requirements concerning implementation
- Select general lifecycle approach in the light of the above.
- Review overall resource estimates
  - Up to this stage,
    - the major risks of the project are identified
    - the overall approach of the project is decided
  - So, it is a good place to re-estimate the required effort and other resources for the project

Step 4: Identify Project Products and Activities

- Identify and describe project products.
  - Identify all the products related to the project.
  - Account for the required activities.
- Document generic product flows
  - To document the relative order of the products.
- Recognize product instances
- Produce an ideal activity network
  - Activity network shows the tasks that have to be carried out as well as their sequence of execution for the creation of a product from another
- Modify the ideal to take into account need for stages and checkpoints.
  - To check compatibility of products of previous activities.

Step 5: Estimate Effort for Each Activity

- Need to estimate staff effort, time for each activity, and other resources
- Need to break a task into a series of manageable sub-tasks

Step 6: Identify Activity Risks

- Identify and quantify the risks of each activity
- Plan risk reduction and likelihood measures where appropriate.
- Adjust overall plans and estimates to take account of the risks.

Step 7: Allocate Resources (Staffing)

- Type of staff needed for each activity are identified
- Staff availabilities are identified, staff are conditionally allocated to task
- Revise plans and estimates to take into account resource constraints

Step 8: Review/publicize Plan

Review quality aspects of the project plan

- To ensure each activity is completed with a quality product

Document plans and obtain agreement

- all parties understand and agree to the commitments in the plan

Step 9: Execute Plan

- Build the physical project deliverables and present them to the customer.
- Once the project is underway, plans will need to be drawn up in greater detail for each activity as it becomes due. Detailed and lower level of planning of the later stages will need to be delayed because more information will be available nearer the start of the stage. Project planning is an iterative process. As the time approaches for the particular activities to be carried out they should be re-planned in more detail.

Stepwise Project Planning

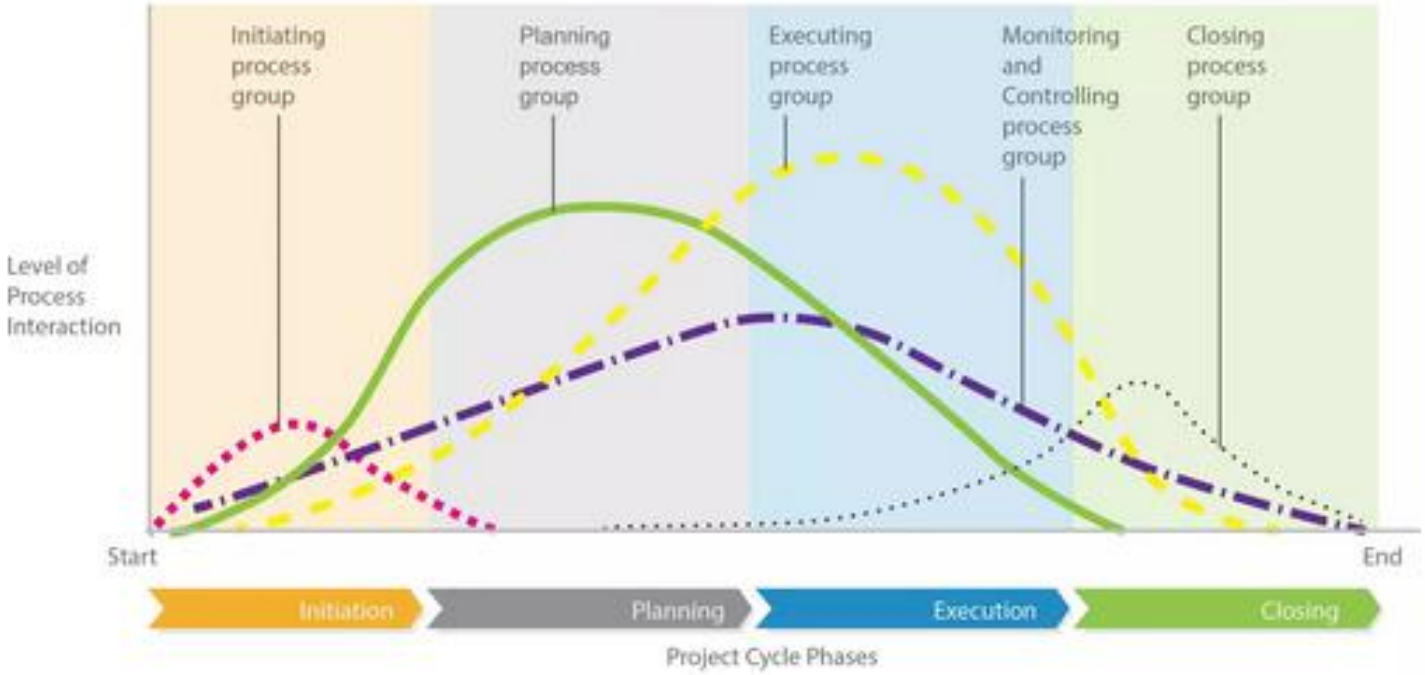
- Step Wise project planning framework
- Preparation of a software project plan
- Planning and scheduling the activities in software project management
- Various approaches towards activity plan
- Various scheduling techniques such as sequencing and CPM

Processes

- Processes are the tasks and methodologies that occur within a phase.
- Although they sound like phases, processes are not phases.
- The PMBOK (Project Management Body of Knowledge) arrange processes into groups according to when they typically occur in a project.
- However, different processes can occur any time during a project phase.

Processes

- This graphic is a representation of when these processes typically occur over the project life cycle



Initiating Process Group

- Typically occurring during the initiation phase of a project.
- These process groups define the new project or phase objectives and what is needed to achieve it.
- Involves obtaining authorization for the project or phase.
- Process group main output is the project charter and stakeholder register.

Initiation processes:

4.1 Develop Project Charter

13.1 Identify Stakeholders

Planning Process Group

- Typically occurring during the planning phase of a project.
- Planning process group does not just apply to the planning phase. Planning is an activity that continues to almost the end of a project, as projects tend to change as it progresses.
- A skilled project manager knows how much time and effort to put into each stage of planning as the project changes

Planning processes:

4.2 Develop Project Management Plan
5.1 Plan Scope Management
5.2 Collect Requirements
5.3 Define Scope
5.4 Create WBS
6.1 Plan Schedule Management
6.2 Define Activities
6.3 Sequence Activities
6.4 Estimate Activity Resources
6.5 Estimate Activity Duration
6.6 Develop Schedule
7.1 Plan Cost Management
7.2 Estimate Costs
7.3 Determine Budget
8.1 Plan Quality Management
9.1 Plan Human Resource Management
10.1 Plan Communications Management
11.1 Plan Risk Management
11.2 Identify Risks
11.3 Perform Qualitative Risk Analysis
11.4 Perform Quantitative Risk Analysis
11.5 Plan Risk Responses
12.1 Plan Procurement Management
13.2 Plan Stakeholder Management

Executing Process Group

- Typically occurring during the implementation/execution phase of a project.
- This process group involves a lot of management tasks and coordination with staff while the project plan is being executed.
- The outputs produced during this process group is the actual project deliverable.

Execution processes:

4.3 Direct and Manage Project Work
8.2 Perform Quality Assurance
9.2 Acquire Project Team
9.3 Develop Project Team
9.4 Manage Project Team
10.2 Manage Communications
12.2 Conduct Procurements
13.3 Manage Stakeholder Engagement

Closing Process Group

- Typically occurring during the closure phase of a project.
- Formal transfer of deliverable to customer
- Administrative closure and contract closure is performed to formally close out project.

Closing processes:

4.6 Close Project or Phase

12.4 Close Procurements

Monitoring and Controlling Process Group

- This process group occurs on all phases of the project life cycle.
- Ensures that the project is on track and flags issues to maintain project course direction.
- Collects performance data through constant reports from team such as status reports and team meetings.

Monitoring and Controlling processes:

4.4 Monitor and Control Project Work

4.5 Perform Integrated Change Control

5.5 Validate Scope

5.6 Control Scope

6.7 Control Schedule

7.4 Control Costs

8.3 Control Quality

10.3 Control Communications

11.6 Control Risks

12.3 Control Procurements

13.4 Control Stakeholder Engagement

From PMBOK, 5th edition

Process Group and Knowledge Area

PROCESS GROUP AND KNOWLEDGE AREA MAPPING

Knowledge Areas	Project Management Process Groups				
	Initiating	Planning	Executing	Monitoring and Controlling	Closing
4. Project Integration Management	4.1 Develop Project Charter	4.2 Develop Project Management Plan	4.3 Direct and Manage Project Work 4.4 Manage Project Knowledge	4.5 Monitor and Control Project Work 4.6 Perform Integrated Change Control	4.7 Close Project or Phase
5. Project Scope Management		5.1 Plan Scope Management 5.2 Collect Requirements 5.3 Define Scope 5.4 Create WBS		5.5 Validate Scope 5.6 Control Scope	
6. Project Schedule Management		6.1 Plan Schedule Management 6.2 Define Activities 6.3 Sequence Activities 6.4 Estimate Activity Durations 6.5 Develop Schedule		6.6 Control Schedule	
7. Project Cost Management		7.1 Plan Cost Management 7.2 Estimate Costs 7.3 Determine Budget		7.4 Control Costs	
8. Project Quality Management		8.1 Plan Quality Management	8.2 Manage Quality	8.3 Control Quality	
9. Project Resource Management		9.1 Plan Resource Management 9.2 Estimate Activity Resources	9.3 Acquire Resources 9.4 Develop Team 9.5 Manage Team	9.6 Control Resources	
10. Project Communications Management		10.1 Plan Communications Management	10.2 Manage Communications	10.3 Monitor Communications	
11. Project Risk Management		11.1 Plan Risk Management 11.2 Identify Risks 11.3 Perform Qualitative Risk Analysis 11.4 Perform Quantitative Risk Analysis 11.5 Plan Risk Responses	11.6 Implement Risk Responses	11.7 Monitor Risks	
12. Project Procurement Management		12.1 Plan Procurement Management	12.2 Conduct Procurements	12.3 Control Procurements	
13. Project Stakeholder Management	13.1 Identify Stakeholders	13.2 Plan Stakeholder Engagement	13.3 Manage Stakeholder Engagement	13.4 Monitor Stakeholder Engagement	

Process Group and Knowledge Area

- While the PMBOK (Project Management Body of Knowledge) and PMI (Project Management Institute) gives an outline of a typical project, no two projects are the same.
- A good project manager knows how to choose which processes are needed for their project.
- Practicality and pragmatism is the best approach to any project. (Keep it simple)

Any Queries?

Project Planning

Unit_3

Project Scope Management

- **Definition:** The processes required to ensure that the project includes **all the work required, and only the work required**, to complete the project successfully.
- **Goal:** To define and control what is and is not included in the project. This prevents **scope creep** (uncontrolled changes or continuous growth in a project's scope) and ensures project success.

Key Processes:

•**Plan Scope Management:**
•Creates the **Scope Management Plan** – how the scope will be defined, validated, and controlled.
•Creates the **Requirements Management Plan** – how requirements will be analyzed, documented, and managed.
•**Collect Requirements:**
•Defining and documenting stakeholders' needs and expectations to meet project objectives.
•**Outputs:** Requirements Documentation, Requirements Traceability Matrix.
•**Define Scope:**
•Developing a detailed description of the project and product.
•**Output: Project Scope Statement** (describes project deliverables, boundaries, and acceptance criteria).

Key Processes:

•**Create WBS (Work Breakdown Structure):**
•**Decomposing** the project scope and major project deliverables into smaller, more manageable components.
•**Output: WBS** (hierarchical decomposition of total scope), **WBS Dictionary** (detailed description of each WBS element).
•**Validate Scope:**
•Formalizing acceptance of the completed project deliverables by the customer or sponsor.
•**Output:** Accepted Deliverables.
•**Control Scope:**
•Monitoring the status of the project and product scope.
•Managing changes to the scope baseline.
•**Output:** Change Requests, Work Performance Information.

Why is it Critical?

- **Clarity:** Ensures everyone understands what needs to be delivered.
- **Control:** Helps prevent unauthorized changes (scope creep).
- **Efficiency:** Focuses resources on necessary work, avoiding wasted effort.
- **Success:** Directly impacts project schedule, cost, and quality objectives.

Example: Community Health Post Construction

Project Goal:

To construct and commission a fully operational, earthquake-resistant, single-story Community Health Post (CHP) in the **Rural Municipality of Myagdi (Province 4)** within 12 months.

1. Define Scope (Project Scope Statement Excerpt):

Element	Description
Project Deliverables	A constructed health post building (2 consultation rooms, 1 pharmacy, 1 waiting area, 2 staff quarters, 2 toilets). Water supply connection (borehole or piped water, including a storage tank). Solar power system (5kW) for lighting and essential equipment. All necessary construction permits from the local *Wada Karyalaya* (Ward Office).
Project Boundaries (Inclusions)	Construction of the physical structure, utility hookups (water/power), and basic fencing. Procurement and installation of basic furniture (desks, chairs, examination couch).
Exclusions (What is NOT in scope)	Procurement of advanced medical equipment (X-ray, ultrasound). Staff recruitment and training (this is an operational scope, not the construction project scope). Construction of an access road (assumed existing path is sufficient).
Acceptance Criteria	Final inspection and approval by the Ministry of Health representative. Issuance of an occupancy certificate from the Rural Municipality Engineer.

2. Create WBS (Work Breakdown Structure) Snippet:

The WBS hierarchically breaks down the scope into manageable work packages.

Level	WBS Element	Type	Example Work Packages
1.0	**Community Health Post Project**	Project	
2.0	**Design & Approvals**	Deliverable	2.1 Topographical Survey (Hills), 2.2 Submit plans to *Wada*, 2.3 Acquire building permit.
3.0	**Site Preparation & Foundation**	Deliverable	3.1 Clear site (remove *jhadi*), 3.2 Excavation, 3.3 Pour plinth beam (earthquake-resistant design).
4.0	**Superstructure Construction**	Deliverable	4.1 Masonry wall construction (*Pathar/Itta*), 4.2 Roof truss assembly, 4.3 Roofing (tin or cement sheet).
5.0	**Utilities & Finishes**	Deliverable	5.1 Plumbing & sanitation, 5.2 Electrical wiring & solar panel install, 5.3 Interior plastering & painting (*Rang-Rogan*).
6.0	**Project Closeout**	Deliverable	6.1 Handover documents, 6.2 Final inspection.

3. Control Scope (Dealing with Scope Creep):

Imagine the following scenario:

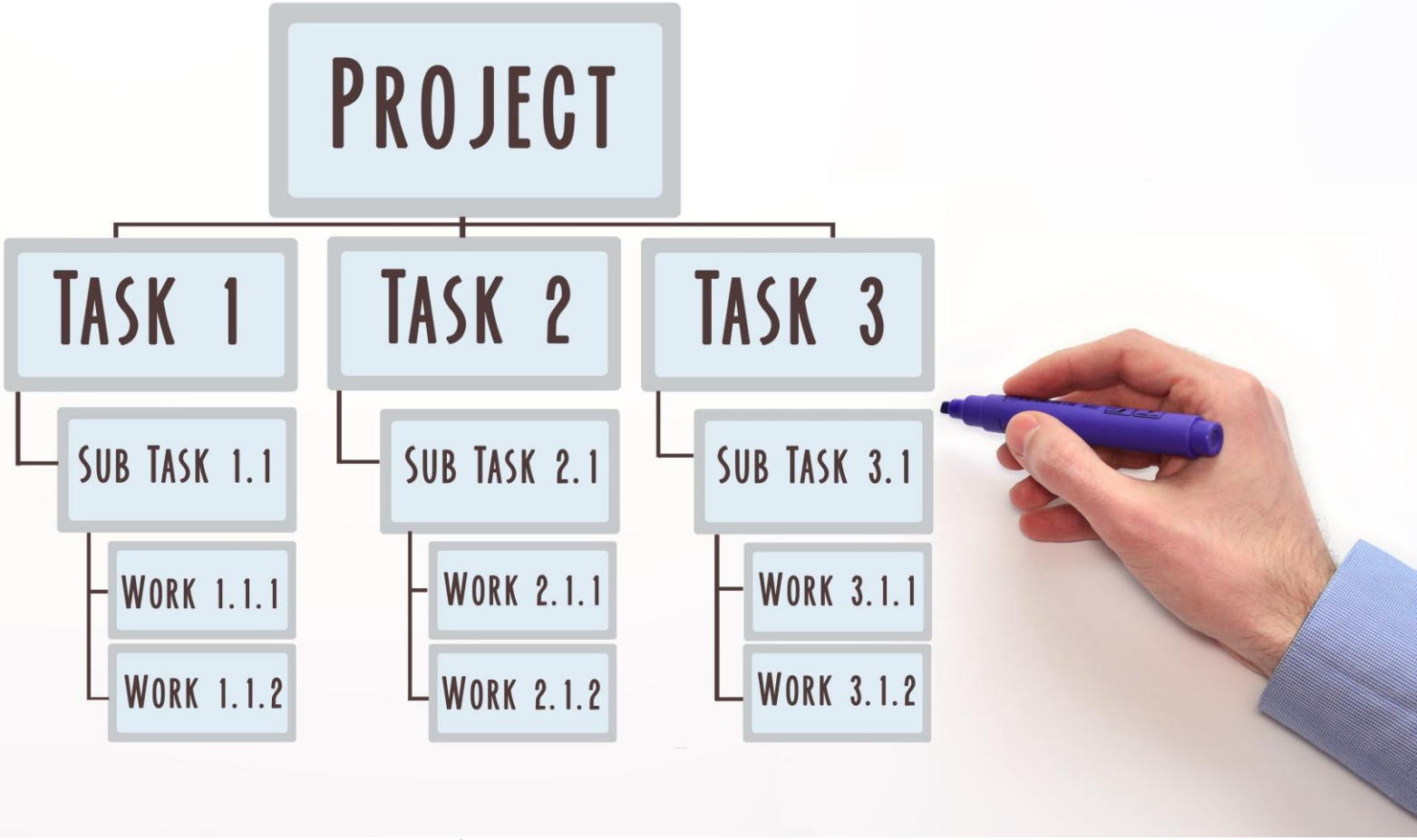
Issue: During construction, the local community elders request that a **separate kitchen/lunch room** be added to the staff quarters, citing long travel times for the nurses.

Project Manager's Response (Using Scope Control):

1. **Identify:** This is a **change request** as it falls outside the original Project Scope Statement and WBS (Exclusions section).
2. **Analyze:** The PM analyzes the impact:
   - **Cost:** Additional material and labor ($1,500 USD equivalent).
   - **Schedule:** 3-week delay to the overall schedule.
   - **Risk:** Requires a minor amendment to the approved building permit.
3. **Action:** The PM submits the request to the Project Sponsor (e.g., NGO head or Ministry official) via the **Change Control Process**.
4. **Decision:** The Sponsor approves the change because the long-term benefit of retaining staff outweighs the cost/schedule impact. The **Scope Baseline** is officially updated to include the new kitchen room.

If the change was denied, the PM would simply inform the stakeholders that the kitchen is **out of scope** for the current project and can be considered for a future phase.

WORK BREAKDOWN STRUCTURE



Work Breakdown Structure (WBS)

- Work Breakdown Structure (WBS) is a hierarchical structure to logically subdivide all the work elements of the project into a graphical presentation
- A project can be logically subdivided by project phases, location, tasks, sub-contractors etc.
- A **deliverable-oriented hierarchical decomposition** of the total scope of work to be executed by the project team.
- Breaks down the project into smaller, more manageable components.
- The lowest level of the WBS is called a **work package**.
- **Key Principle:** The WBS captures **100% of the project scope**, including all deliverables, internal and external.

Importance

- **Clarity & Understanding:** Provides a clear, visual representation of all project deliverables, ensuring everyone understands the full scope.
- **Scope Definition:** Helps define what is *in* and *out* of scope, preventing **scope creep**.
- **Improved Planning:** Facilitates detailed planning for schedule, cost, resources, and risks at a granular level.
- **Responsibility Assignment:** Easier to assign clear ownership and accountability for specific work packages.
- **Progress Tracking:** Enables more accurate monitoring and reporting of project progress against defined deliverables.
- **Estimation:** Allows for more precise cost and duration estimates for each work package.

Organizing a Cultural Event (e.g., Dashain Fair)

- Imagine a project to organize a **Dashain Cultural Fair** in a community ground in Kathmandu.

Understanding the WBS Levels:

- **Level 1 (Project):** "Dashain Cultural Fair" - The overall project.
- **Level 2 (Major Deliverables):** "Venue Setup," "Entertainment," "Food & Stalls," "Marketing," "Logistics" - Major components required for the fair.
- **Level 3 (Sub-Deliverables/Activities):**
  - Under "Venue Setup": "Stage Construction," "Seating Arrangement," "Decorations."
  - Under "Entertainment": "Cultural Dance," "Music Performances," "Kids Zone."
- **Level 4 (Work Packages):** The lowest level, representing specific, manageable tasks that can be assigned, estimated, and tracked.
  - Under "Stage Construction": "Order Materials," "Assemble Stage Frame," "Install Lighting."
  - Under "Cultural Dance": "Hire Dance Troupes," "Rehearsal Schedule."

Each work package has a clear deliverable. For example, "Assemble Stage Frame" delivers a complete stage frame.

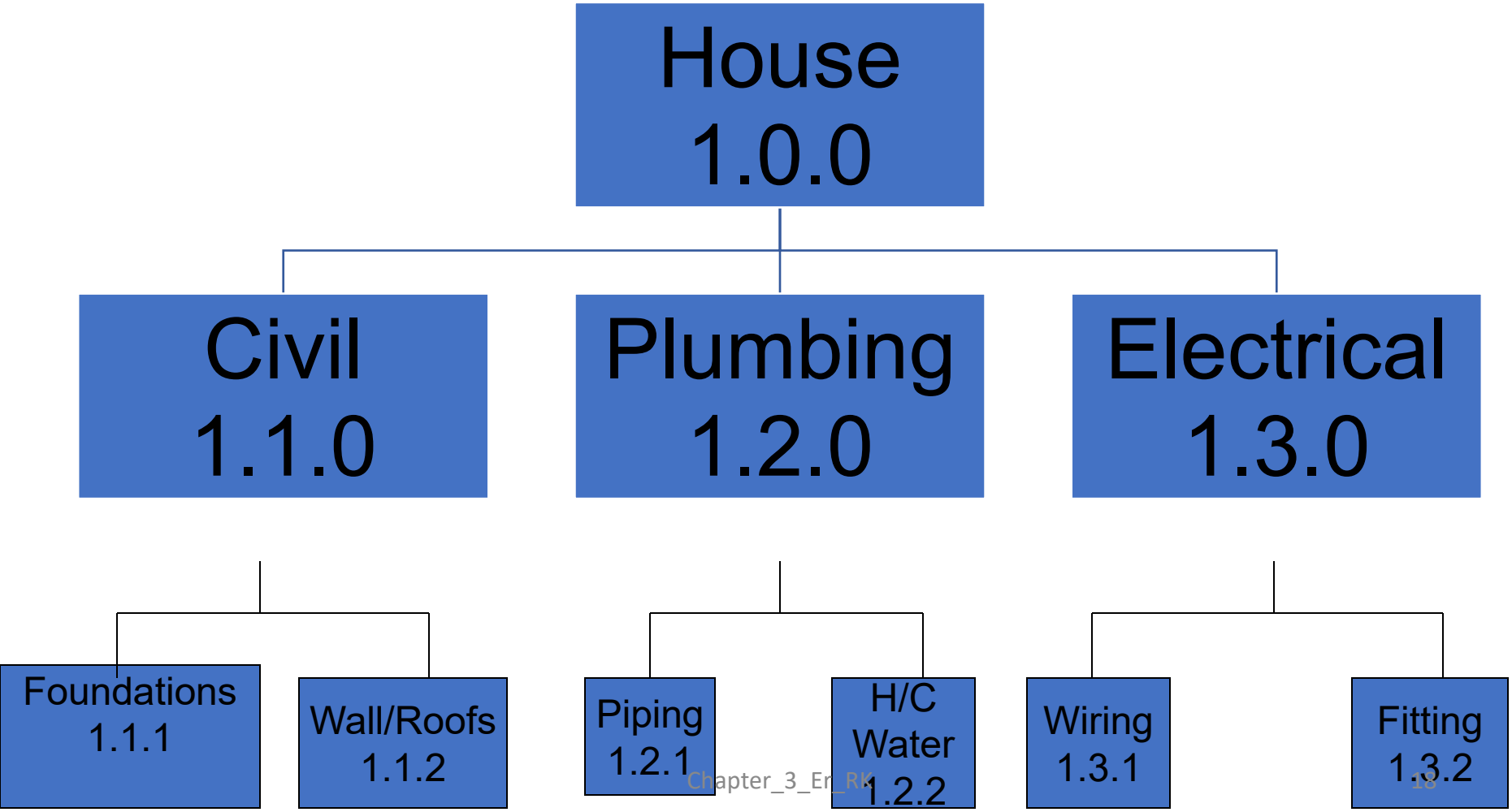
Components of Work Breakdown Structure

- Structure
- Description
- Coding
- Number of WBS Levels
- The level of details
- Roll-up

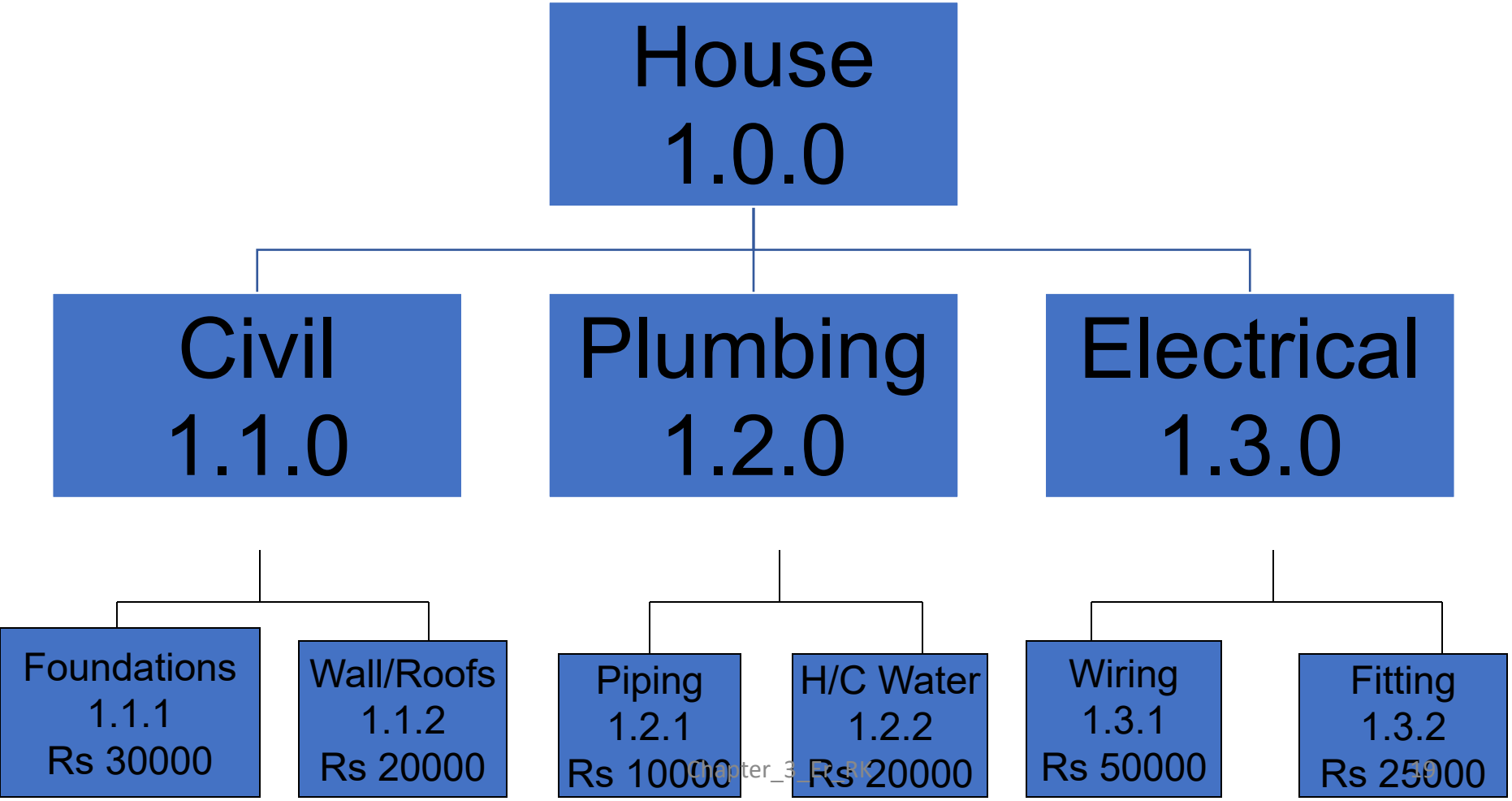
Uses of WBS

- It helps to visualize the scope of project clearly
- It helps to develop a complete lists of activities of the project
- It helps to estimate budget for the project

Work Breakdown Structure and Description

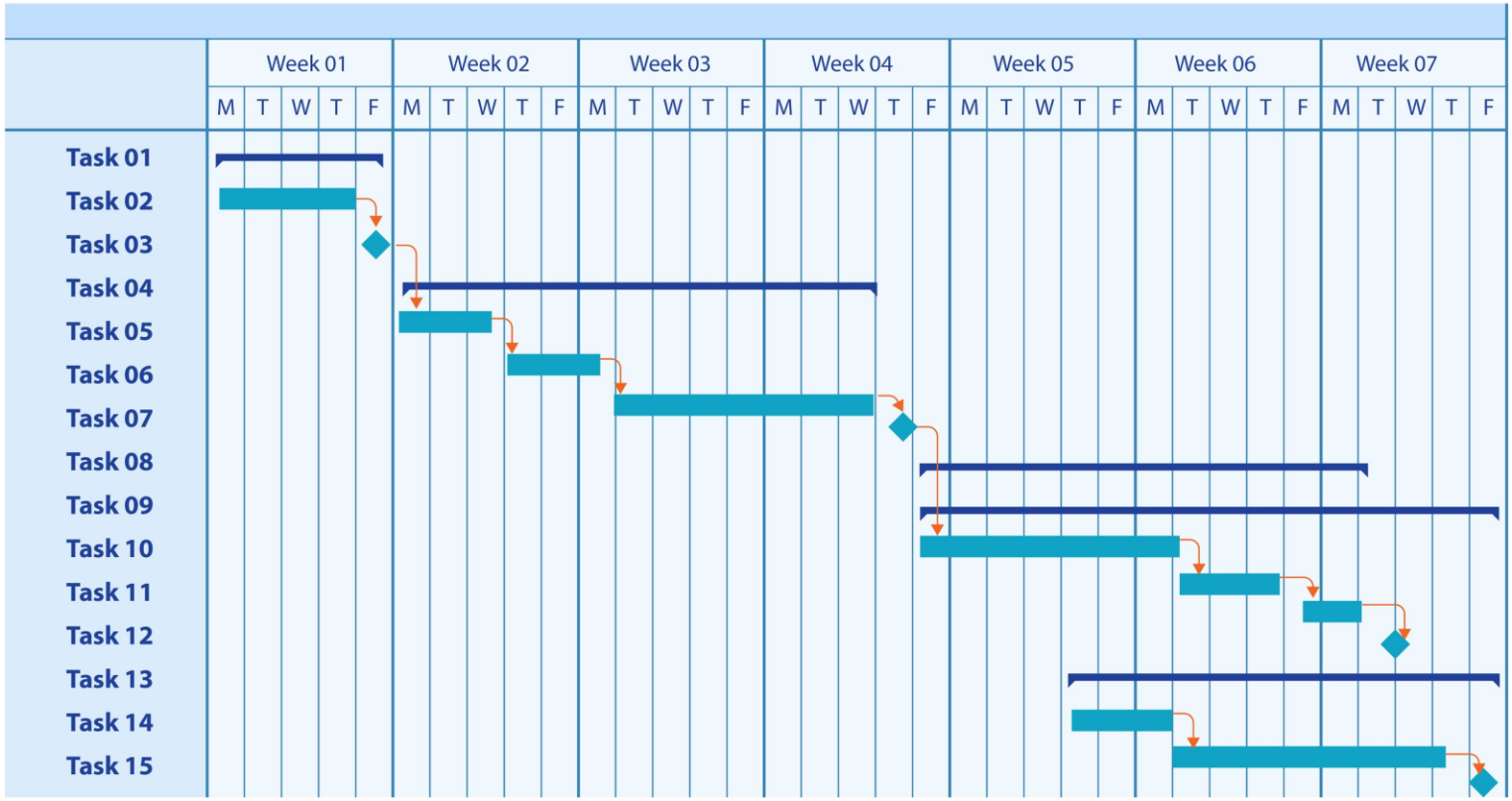


Work Breakdown Structure and Description



Time Management

- This involves the processes required to manage the **timely completion of the project**.
- **Scheduling:** Determining when project activities will be performed.
- **Gantt Charts:** A **bar chart** that shows the timeline for a project schedule, displaying the start and finish dates of the various elements.
- **Network Diagrams:** A graphical representation of the logical relationships (dependencies) among the project schedule activities. They show the sequence of work.
- **CPM (Critical Path Method):** A technique used to determine the **longest sequence of activities** in a project plan, which determines the shortest possible time to complete the project. The activities on this path are the **critical path** activities.
- **PERT (Program Evaluation and Review Technique):** A method for analyzing the tasks involved in completing a given project, especially useful when the time required for activities is uncertain. It uses optimistic, pessimistic, and most likely time estimates to calculate expected duration.


Week 01
Week 02
Week 03
Week 04
Week 05
Week 06
Week 07
M T W T F
Task 01
Task 02
Task 03
Task 04
Task 05
Task 06
Task 07
Task 08
Task 09
Task 10
Task 11
Task 12
Task 13
Task 14
Task 15

Project Cost Management

- Project cost management ensures that the work is completed within the approved financial resources. It involves estimating all expenses, preparing a budget, and controlling costs throughout the project.
- It answers three essential questions:

1. How much will the project cost
2. When will the money be spent
3. How do we prevent overspending

Cost Management

- This includes the processes involved in **planning, estimating, budgeting, financing, funding, managing, and controlling costs** so that the project can be completed within the approved budget.
- **Budgeting:** Aggregating the estimated costs of individual activities or work packages to establish an authorized cost baseline.
- **Estimation Techniques:** Methods like **Analogous Estimating** (using past similar projects), **Parametric Estimating** (using mathematical models), and **Three-Point Estimating** (like PERT, using O, P, M estimates).

- Cost estimation is the process of predicting the financial resources required to complete all project activities. These include:
  - Labour
  - Materials
  - Equipment
  - Software and hardware
  - Travel
  - Training
  - Contingency reserves
  - Overheads and administrative costs
- A clear estimation supports planning, decision-making, and budgeting.

Common Estimation Techniques

A. Analogous Estimation

B. Parametric Estimation

C. Bottom-up Estimation

D. Three-Point Estimation (PERT Cost Method)

E. Expert Judgment

F. Vendor Bid / Tender Estimation

A. Analogous Estimation

- This method uses the cost of a similar past project as a reference.
- **Example**
  - A private IT firm in Kathmandu developed a school information system last year for NPR 8,00,000.
  - A new school in Pokhara now wants a similar system. Because the size and features are nearly identical, the team may estimate the new project to cost around the same amount, perhaps adding a small adjustment for inflation or added features.
- **Strength**
  - Quick and simple
- **Limit**
  - Less accurate because it depends on similarity.

B. Parametric Estimation

- Uses measurable units and applies a cost-per-unit factor.
- **Example**
  - A municipality plans to digitize 20,000 citizen records. If the cost to digitize one record is NPR 12 (from previous projects):
  - Calculation: 20,000 × NPR 12 = **NPR 2,40,000**
- This estimation works well when the work is repeatable and the per-unit cost is known.

C. Bottom-up Estimation

- Breaks the project into small work packages and estimates each item separately. All costs are then combined to find the total.
- Example: E-library System for a Rural Municipality

Work Package	Estimated Cost (NPR)
Requirement collection	40,000
UI/UX design	60,000
Backend development	1,50,000
Frontend development	1,20,000
Testing	50,000
Deployment	30,000
Training	20,000

Total project cost = NPR 4,70,000

This method is more accurate and is often used in formal contracts in Nepal.

D. Three-Point Estimation (PERT Cost Method)

Uses three values:

- **Optimistic (O)**
- **Most likely (M)**
- **Pessimistic (P)**

Expected cost is:

$$\text{Expected Cost} = \frac{O + 4M + P}{6}$$

Example

Cost of developing a new module for a hospital system:

- O = 80,000
- M = 1,00,000
- P = 1,60,000

$$\frac{80,000 + 4(1,00,000) + 1,60,000}{6} = \frac{80,000 + 4,00,000 + 1,60,000}{6} = \frac{6,40,000}{6} \approx 1,06,667$$

Expected cost ≈ **NPR 1,06,667**.

This helps when there is uncertainty or changing requirements.

E. Expert Judgment

- Cost is estimated using the experience of senior professionals who have handled similar projects.
- **Nepal Example**
- A government tender for a digital land-record system may rely on experts who understand software development, local infrastructure, and hardware procurement.

F. Vendor Bid / Tender Estimation

- Used when vendors submit financial proposals for a project.
  The project team compares bids and selects the most suitable offer based on **cost, quality, and capability**.
- **Example**
- The Nepal government's IT projects such as Nagarik App improvements or online payment gateways are often awarded through competitive bidding.

2. Budgeting

- Budgeting assigns estimated costs to project activities and produces the total approved project cost.
- It also determines when funds will be required at each stage.
- A proper budget includes:
  1. Activity cost estimates
  2. Contingency reserves (for known risks)
  3. Management reserves (for unknown risks)
  4. Cash-flow plan
  5. Cost baseline

Budgeting Example (Nepal Context)

A college in Pokhara plans to build a **Computer Lab**.

Category	Cost (NPR)
Computers (20 units × 45,000)	9,00,000
Furniture (tables, chairs)	1,50,000
Networking equipment	80,000
Electrical wiring	70,000
Software licenses	1,20,000
Labour and installation	60,000
Contingency reserve (10 percent)	1,08,000

Total Estimated Budget
NPR 15,88,000
The college prepares this as the cost baseline.
Monthly or weekly expenditure is then matched to the budget to monitor spending.

3. Cost Control

- Cost control ensures that the project does not exceed the budget.
  It includes monitoring spending, updating forecasts, approving variations, and reporting cost performance.
  - Common tools:
  - Earned Value Management
  - Cost variance analysis
  - Forecasting (Estimate at Completion)
  - Cost monitoring dashboards
- **Example**
- During the construction of an ICT lab, unexpected price increases in hardware may require a revised budget and approval from the college board.

Validating Examples (Simple Cases)

- **Example 1: Comparing Estimation Techniques**
- A school wants a small website.
- Analogous: Last year's website cost NPR 50,000 → estimate same.
- Parametric: 10 pages × NPR 4,000 per page = NPR 40,000
- Bottom-up:
  - Design: 10,000
  - Development: 22,000
  - Hosting: 5,000
  - Testing: 4,000
    Total = **NPR 41,000**
- Bottom-up is closest to the real value.

- **Example 2: Budget Revision Scenario**
- A municipality planned NPR 4,70,000 for an e-library.
  A new requirement added "SMSNotifications."
- Added cost: NPR 35,000.
- New budget
- 4,70,000+35,000=5,05,000
- Scope change increases the budget.

Risk Management

- Risk management is the process of identifying uncertain events that may affect a project, analyzing their impact, and taking actions to reduce their negative effects. It ensures that a project is prepared for unexpected situations.
  - A risk may cause:
  - Delay in schedule
  - Increase in cost
  - Reduction in quality
  - Failure to meet objectives
- Effective risk management increases the chance of project success.

1. Risk Identification

- Risk identification is the process of listing all possible risks that could harm the project. The goal is not to solve risks immediately, but to recognize them clearly.
- **Methods Used in Risk Identification**
  - Brainstorming
  - Expert interview
  - Checklists
  - Past project study
  - SWOT analysis (Strengths, Weaknesses, Opportunities, Threats)

Types of Risks to Identify

- **Technical Risks**
  - Failure of technology, performance issues, integration problems
- **Organizational Risks**
  - Staff shortage, poor coordination, change in leadership
- **External Risks**
  - Government policy changes, natural disasters, strikes
- **Project Management Risks**
  - Poor planning, unclear requirements, scope creep

Examples

- **Example 1: Hospital Management System Development (Nepal)**
- Possible risks identified:
- Delay in requirement approval from hospital administration
- Internet failure during system deployment
- Software integration issues with old hardware
- Sudden increase in patient data load beyond server capacity

- **Example 2: Construction of ICT Lab in a Public School**
- Risks:
- Suppliers may delay hardware delivery
- Electrical wiring may fail inspection
- Budget shortage
- Exams or festivals causing interruption in schedule

2. Risk Analysis

- Risk analysis evaluates the likelihood of each risk and the severity of its impact. The purpose is to prioritize which risks need immediate attention.
- There are **two main methods**:
  - A. Qualitative Risk Analysis
  - B. Quantitative Risk Analysis

A. Qualitative Risk Analysis

- Classifies risks as:
  - High
  - Medium
  - Low
- Based on probability and impact.

Example

Project: Online Municipal Tax System

Risk	Probability	Impact	Priority
Delay in government approval	High	High	High
Internet outage	Medium	Medium	Medium
Developer leaving job	Low	High	Medium
Data entry mistakes	Medium	Low	Low

High-priority risk requires immediate planning.

B. Quantitative Risk Analysis

- Uses numerical values to calculate the expected impact of risks.
  - Common tools:
  - Probability × Impact formula
  - Sensitivity analysis
  - Expected Monetary Value (EMV)

Example of Quantitative Analysis

A software server upgrade project identifies a risk:

•Probability of hardware failure: 20 percent

•Impact cost if failure occurs: NPR 3,00,000

Expected Monetary Value (EMV):

$$EMV = 0.20 \times 3{,}00{,}000 = 60{,}000$$

This means the project should reserve **NPR 60,000** to cover this risk.

3. Risk Mitigation Strategies

- Mitigation strategies reduce the impact or likelihood of risks.
- There are **four main response strategies**.
  - A. Risk Avoidance
  - B. Risk Reduction (Mitigation)
  - C. Risk Transfer
  - D. Risk Acceptance

- **A. Risk Avoidance**
- Eliminating the risk by changing the plan.
- **Example**
- If a web application feature depends on unstable third-party software, the project may decide to remove or replace that feature entirely.
- A municipality avoids using a small local SMS provider because of frequent outages. Instead, they use a more stable national provider.

- **B. Risk Reduction (Mitigation)**
- Taking actions to decrease the probability or impact.
- **Example**
- Adding additional testing to reduce the chance of software defects.
- To reduce the risk of power failure during deployment in rural schools, the team carries backup batteries and UPS systems.

- **C. Risk Transfer**
- Shifting the risk to another party, usually through contracts or insurance.
- **Example**
- A company outsources server hosting to a certified cloud vendor who guarantees uptime.
- A bank transfers cyber risk to a security firm responsible for 24-hour monitoring.

- **D. Risk Acceptance**
- The team acknowledges the risk and decides to deal with it if it occurs.
- Used when the cost of mitigation is higher than the risk impact.
- **Example**
- Minor graphical issues in the user interface are accepted because they do not affect system functionality.
- A small local startup accepts the risk of a slight delivery delay due to festival traffic.

Example of Full Risk Management Process

- **Project:** Deploying a Learning Management System (LMS) in a Pokhara College

- **Step 1: Identify Risks**
- Staff may resist using the new system
- Server may fail during peak load
- Internet connection may be unstable
- Extra training sessions may be required

Step 2: Analyze Risks

Risk	Probability	Impact	Priority
Server failure	Medium	High	High
Staff resistance	High	Medium	High
Internet issues	High	High	High
Extra training required	Medium	Low	Medium

Step 3: Mitigation Strategies

- Upgrade server capacity or use cloud hosting (Risk Reduction)
- Conduct early orientation for staff (Risk Reduction)
- Install backup internet connection (Risk Avoidance)
- Accept possible need for extra training (Risk Acceptance)

Communication Management

- Communication management ensures that the right information reaches the right people at the right time. It helps maintain coordination, reduce misunderstandings, and build trust among everyone involved in a project.
- A project fails more often because of poor communication than because of technical problems.

1. Stakeholder Engagement

- Stakeholder engagement is the process of identifying all individuals or groups that have an interest in the project and keeping them involved through proper communication.
- **Stakeholders may include:**
  - Project sponsor
  - Project manager
  - Team members
  - Government bodies
  - Clients and end users
  - Vendors and suppliers
  - Local communities (for public projects)
- The goal is to manage expectations, reduce conflict, and ensure everyone understands their responsibilities.

Steps in Stakeholder Engagement

- **A. Identify Stakeholders**
- Create a list of all people affected by the project.
- **Example :**
- Developing a digital registration system for a rural municipality.
  - Stakeholders include:
  - Municipal IT officer
  - Ward secretaries
  - Citizens using the system
  - Software developers
  - Data entry staff

Steps in Stakeholder Engagement

- **B. Analyze Stakeholder Interests**
- Not all stakeholders have the same influence or concern.
- Some may focus on cost, others on usability or security.
- Use a simple grid:

Stakeholder	Interest	Influence	Priority
Municipal Chief	System reliability	High	High
IT Officer	Technical performance	Medium	High
Citizens	Ease of use	High	Medium

Steps in Stakeholder Engagement

- **C. Plan How to Engage Stakeholders**
- Decide how often to communicate and what information to share.
- Examples:
  - Weekly meetings with the municipality
  - Monthly progress reports for the mayor
  - Daily internal updates for developers

Steps in Stakeholder Engagement

- **D. Manage and Monitor Engagement**
- As the project progresses, keep reviewing stakeholder needs.
- If expectations change, update plans accordingly.
- **Example:**
- Teachers in a Pokhara school request extra training when using a new LMS.
- The project team updates the schedule and adds a training session.

Example of Stakeholder Engagement

- **Case: ICT Lab Setup in a Public School**
  - Principal wants timely completion
  - Students want good-speed computers
  - Teachers want training sessions
  - Suppliers want clear payment schedules
  - Local government wants compliance with budget rules
- Engagement actions:
  - Weekly coordination meetings
  - Sharing installation plans with teachers
  - Providing a training workshop
  - Regular status updates to the municipality
- This prevents misunderstandings and keeps everyone aligned.

2. Information Distribution

- Information distribution is the process of sharing the right information with the right stakeholders using the right tools.
- The goal is to ensure transparency, smooth progress, and strong teamwork.

Methods of Information Distribution

- **A. Meetings**
- Daily stand-up meetings for the internal team
- Weekly review meetings with clients
- Monthly oversight meetings with government officials (if required)
- **Example:**
- In a software project for a hospital, the developer team meets every morning, while the hospital management receives a progress update every Friday.

Methods of Information Distribution

- **B. Reports**
- Reports summarize progress, risk status, budget updates, and upcoming tasks.
- Common project reports include:
  - Weekly status report
  - Monthly progress report
  - Risk report
  - Budget consumption report
- **Example:**
- A construction company working on a college building sends a monthly progress report to the college board with photos and financial updates.

Methods of Information Distribution

- **C. Communication Tools**
- These tools help share documents, updates, and messages.
- Examples:
  - Email
  - Zoom or Google Meet
  - Slack or Microsoft Teams
  - Project management tools (Trello, Jira, Asana)
  - WhatsApp for fast communication (widely used)
- **Example:**
- Developers working remotely from Kathmandu and Pokhara use Slack channels for code reviews and GitHub for file sharing.

Methods of Information Distribution

- **D. Documentation and Record Keeping**
- All meeting notes, design documents, contracts, and change requests must be recorded.
- This prevents disputes.
- **Example :**
- A municipality project stores all meeting minutes and agreements in a shared Google Drive folder and keeps physical copies for government audit.

Methods of Information Distribution

- **E. Dashboards and Visual Charts**
- Dashboards provide real-time project status through graphs and indicators.
- Useful for large projects with many stakeholders.
- **Example:**
- A hydropower project uses a dashboard showing construction progress and expenses so that the Ministry can monitor it remotely.

Example

- **Project: Developing an Online Entrance Registration System for a College**
- **Stakeholder Engagement Activities:**
  - Identify stakeholders: College management, IT staff, applicants, admission committee
  - Conduct initial meeting to collect expectations
  - Prepare a communication plan for updates
  - Schedule demonstration sessions for the committee
- **Information Distribution Activities:**
  - Weekly progress report to the college
  - Email notification to applicants during testing phase
  - Internal daily updates for the development team
  - Change requests recorded in Google Sheets
  - Training manual shared as PDF to staff
- This ensures the project moves smoothly without confusion.

Working Environment in Project Management

- The working environment describes **where project activities are performed** and **how teams collaborate**.
- The environment affects communication, productivity, cost, and project timelines.
- There are three common working arrangements:
  - **Onsite**
  - **Offsite**
  - **Work From Home (WFH)**
- Each environment has benefits and challenges.

1. Onsite Working Environment

- In an onsite setup, the project team works **physically at the client's or organization's location**.
- **Characteristics**
  - Direct access to clients, data, and equipment
  - Faster decision-making
  - Easier supervision and coordination
  - Suitable for projects requiring physical presence, such as installation, testing, or fieldwork

1. Onsite Working Environment

- **Advantages**
  - Immediate feedback from stakeholders
  - Strong teamwork
  - Better observation of real working conditions
  - Faster troubleshooting
- **Limitations**
  - Travel cost and travel time
  - Less flexibility
  - Limited working hours
  - Stress for team members if location is far or crowded

1. Onsite Working Environment

- **Example**
- A team installing a **Hospital Management System (HMS)** in a government hospital must work **onsite** to configure servers, train staff, and test real-time operations.

2. Offsite Working Environment

- Offsite work means the team works **from a different location**, usually from their own office or a dedicated project center, not at the client's site.
- **Characteristics**
  - Used when onsite access is not necessary
  - Communication happens through periodic meetings, emails, and online tools
  - Vendors can serve clients in different cities or countries

2. Offsite Working Environment

- **Advantages**
  - Lower travel costs
  - Quiet and controlled environment
  - Higher flexibility in planning team tasks
  - Access to internal company resources and support teams
- **Limitations**
  - Slower communication with the client
  - Difficult to understand client's day-to-day operations
  - Problems may take longer to resolve
- **Example**
  - A software company in Kathmandu develops an **Online Banking App for a Pokhara-based bank**. Most work (coding, design, testing) is done **offsite**, with meetings scheduled online.

3. Work From Home (WFH)

- In this arrangement, team members work from their homes using laptops and communication tools.
- WFH became popular globally due to COVID-19 and continues to be widely used in IT and research-based work.
- **Characteristics**
  - Employees work remotely
  - Work is coordinated through online tools and cloud systems
  - Suitable for coding, documentation, design, analysis, content creation

3. Work From Home (WFH)

- **Advantages**
  - Flexible working hours
  - Saves travel expenses and time
  - Helps maintain work-life balance
  - Useful for employees living far from office locations
- **Limitations**
  - Requires stable internet connection
  - Communication may be slower
  - Harder to monitor productivity
  - Lack of teamwork feeling due to isolation
- **Example**
  - During the COVID-19 lockdown, many IT firms in Lalitpur, Pokhara, and Bharatpur completed **mobile app development** and **digital services** through WFH.
  - Team members used Zoom, GitHub, Slack, and Google Drive to share work.

Comparative Table

Factor	Onsite	Offsite	Work From Home
Communication	Fast, direct	Good but scheduled	Mostly digital
Flexibility	Low	Medium	High
Cost	High (travel, setup)	Medium	Low
Monitoring	Easy	Moderate	Difficult
Suitable for	Testing, installation	Development, analysis	Coding, documentation

Example

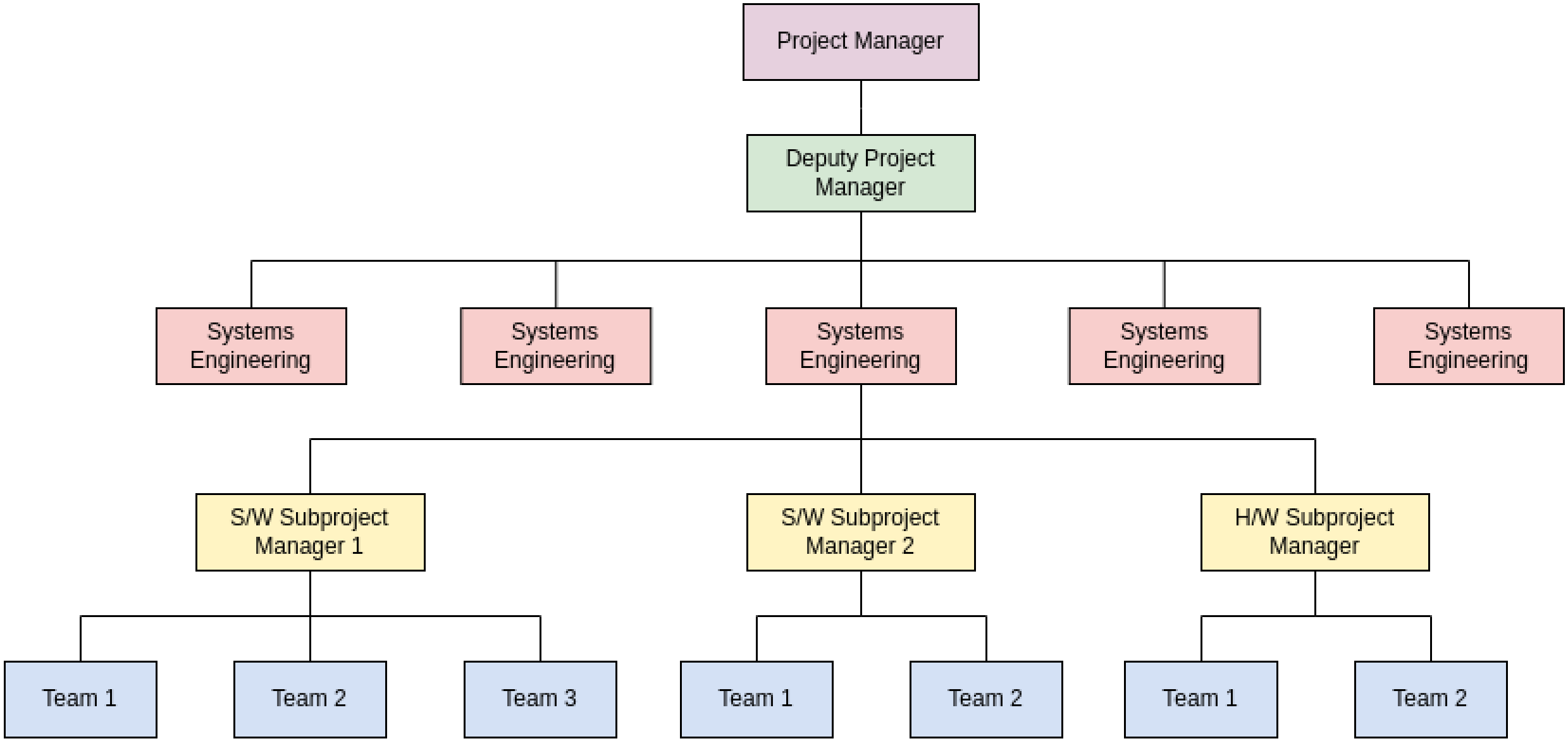
- **Project: Developing and launching an E-learning System for a college in Pokhara**
- **Onsite Work**
  - Team visits the college to collect requirements
  - Tests the system with teachers and students
  - Conducts training sessions in the computer lab
- **Offsite Work**
  - Coding and UI design done at the company office in Kathmandu
  - Backend testing and database setup done in the development center
- **Work From Home**
  - Documentation writer prepares manuals from home
  - Part-time designer creates graphics remotely
  - Evening bug fixes done by remote developers
- This combination creates efficiency while managing cost and team comfort.

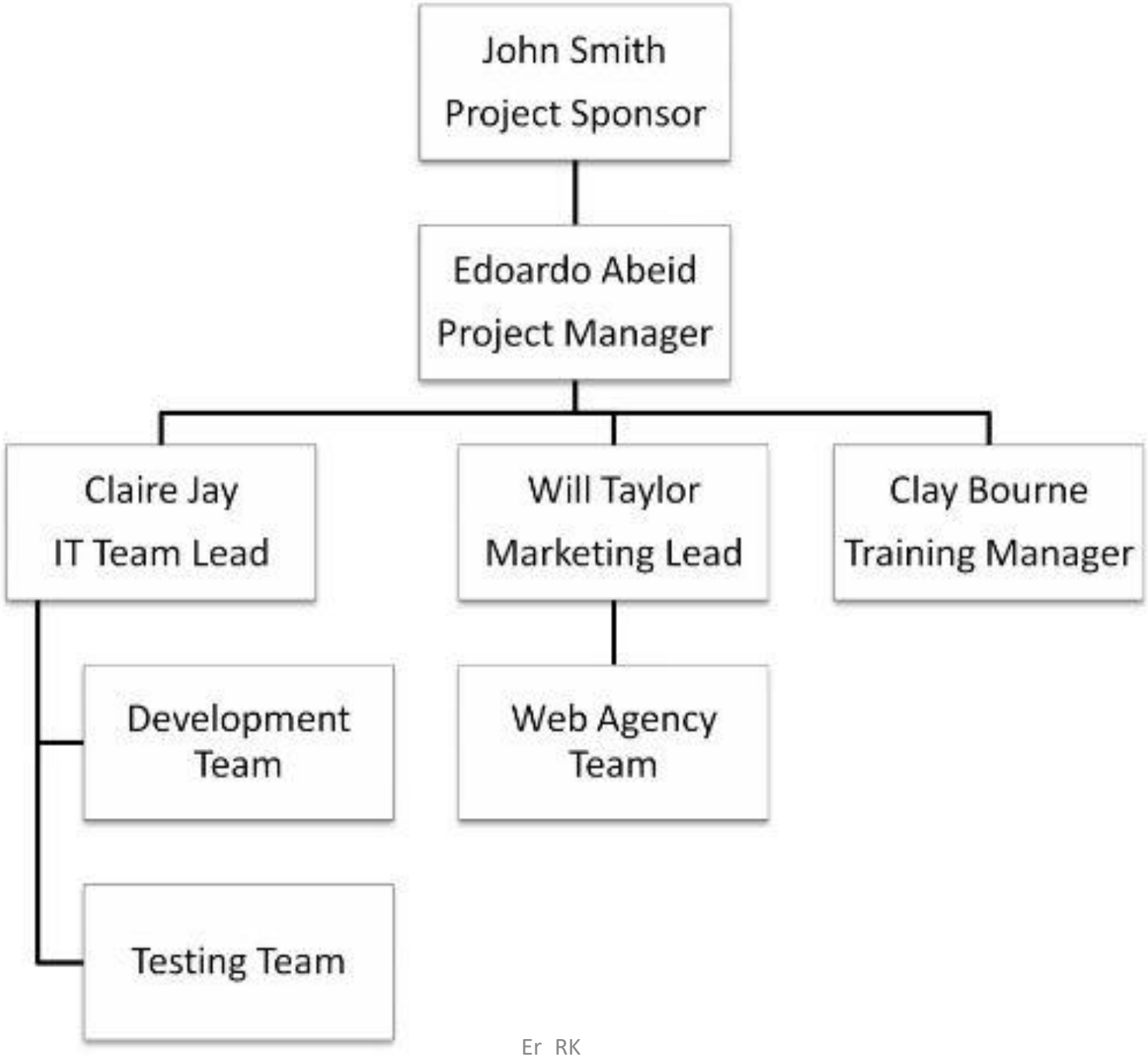
Gantt Charts

- A Gantt chart is a horizontal bar chart that represents a project schedule.
- It is a favored tool in project management for scheduling tasks over the duration of the project.

Project Organization and Responsibilities

Unit_4


Project Manager
Deputy Project Manager
Systems Engineering
Systems Engineering
Systems Engineering
Systems Engineering
Systems Engineering
S/W Subproject Manager 1
S/W Subproject Manager 2
H/W Subproject Manager
Team 1
Team 2
Team 3
Team 1
Team 2
Team 1
Team 2


John Smith
Project Sponsor
Edoardo Abeid
Project Manager
Claire Jay
IT Team Lead
Will Taylor
Marketing Lead
Clay Bourne
Training Manager
Development Team
Web Agency Team
Testing Team

Meaning of Project Organization

- Project organization defines how people, roles, and reporting lines are arranged so that a project can be planned, executed, and completed in a controlled manner.
- A well-designed structure prevents overlap, avoids delays, and assigns clear accountability.
- A project is temporary, so its organization focuses on efficiency, clarity, and coordination rather than long-term hierarchy.

Meaning of Project Organization

- Project organization refers to the way people, roles, and responsibilities are arranged to complete a project.
- It helps define **who does what**, **how decisions are made**, and **how information flows** within a project.
- **Why Project Organization Is Needed?**
  - Ensures clear responsibility
  - Reduces confusion during work
  - Improves coordination between teams
  - Helps monitor progress and solve problems quickly
  - Supports timely decision-making

2. Major Roles and Their Responsibilities

- **A. Project Sponsor**
- The sponsor is the senior authority who approves the project and supports it throughout.
- **Key responsibilities**
  - Sanctions budget and resources
  - Approves major changes
  - Clears obstacles that the project manager cannot solve
  - Ensures the project supports overall organizational goals
- **Example for validation**
  - A hydropower project in Nepal (such as a 25 MW project funded by a local investor group) needs approval for tunnel construction. The sponsor ensures the funds are available and approves the engineering changes. Without the sponsor, the project manager cannot move ahead with procurement or major structural adjustments.

- **B. Project Manager**
  - The project manager coordinates the work from start to finish.
- **Key responsibilities**
  - Creates the project plan
  - Assigns tasks to team members
  - Manages cost, time, and scope
  - Conducts risk analysis and mitigation
  - Communicates with stakeholders
  - Ensures the work meets quality expectations
- **Example for validation**
  - In a road construction project under the Department of Roads, the project manager plans the schedule for surveying, design, and construction. If heavy rainfall delays excavation, the manager adjusts the timeline and reallocates resources to maintain progress. This illustrates active control of time and resources.

- **C. Project Team Members**
- These are specialists or workers who perform the actual tasks.
- **Responsibilities**
- Complete assigned activities
- Report progress and issues
- Follow quality and safety standards
- Cooperate with other teams
- **Example**
- During the development of a mobile banking application in Nepal, team programmers build specific modules such as login, fund transfer, and QR payments. One developer identifies a security loophole and reports it early, preventing later delays. This shows the importance of responsibility and early communication.

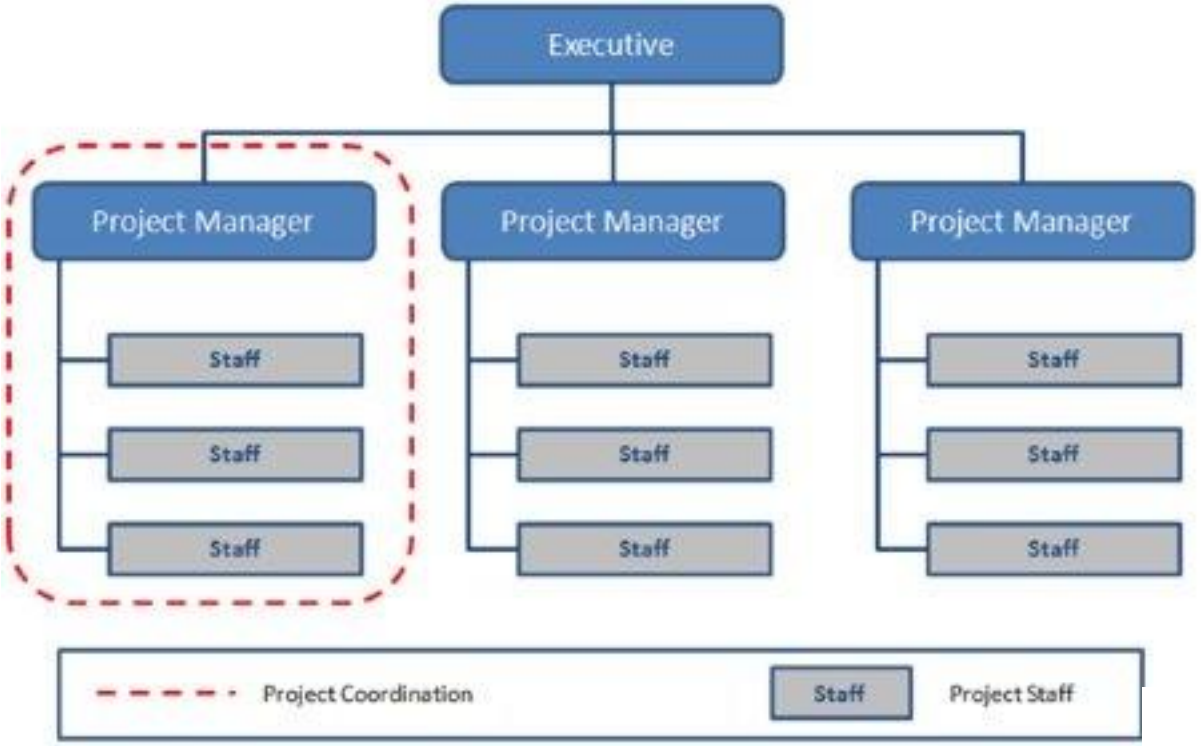
- **D. Stakeholders**
- Stakeholders include individuals, groups, or institutions that affect or are affected by the project.
- **Responsibilities**
- Provide requirements
- Give feedback
- Monitor compliance with regulations
- Approve outputs when necessary
- **Example**
- A hospital expansion project in Pokhara requires approval from the local municipality, the health ministry, and nearby residents affected by traffic. Their role is not technical work but coordination, review, and permission. This confirms that stakeholders help shape and validate project decisions.

- **E. Support Functions**
- These are units that help the project operate smoothly.
- **Common support groups**
- Finance (budget handling)
- HR (recruitment and contracts)
- Procurement (materials and vendors)
- IT support (systems and tools)
- Legal and compliance
- **Example**
- In a software project for a private college in Nepal, the procurement unit handles hardware purchases for servers. This keeps the project team focused on development rather than administrative tasks.

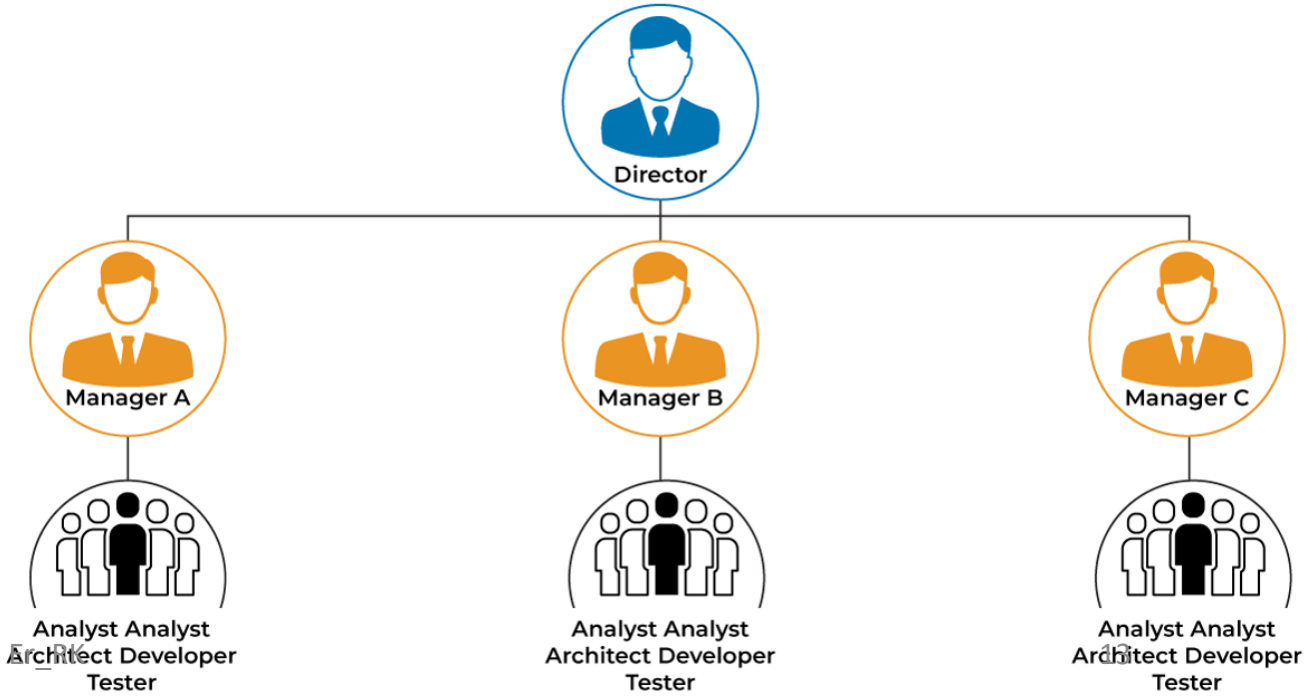
Types of Project Organization Structures

- When designing a project organization inside an existing institution, the most important factor is **how much authority top management is willing to give to the project manager**.
- Top management decides:
  - What the project manager can approve
  - How teams will be formed
  - How responsibilities will be divided
  - How communication will flow
  - How conflicts between project and departments will be resolved

- Development organizations (such as NGOs, government programs, health or education sectors) usually work in **program units**. Within these units, three project organization choices are available:
  1. **Programmatic Based**
  2. **Matrix Based**
  3. **Project Based**
- Each structure is defined by the *level of authority given to the project manager*.



Programmatic Based Organization Structure



Programmatic Based Organization

- A programmatic structure is the traditional arrangement where project resources come from one program sector (such as health, education, agriculture, etc.).
- The project manager has authority only **within that program area**.
- Example:
- A **health-focused NGO** places a maternal nutrition project inside its Health Unit.
- All staff (nutritionists, field workers, trainers) come from the same unit.

Characteristics

- Project teams are staffed only from one program sector
- Team members usually know each other and share similar skills
- Clear reporting lines
- Project manager often doubles as program unit manager

- **Advantages**

1. **Clear authority**
    1. The program manager controls all resources needed for the project.
2. **Easy coordination**
    1. Since staff belong to the same department, communication is simple.
3. **Shared expertise**
    1. Members already understand the subject area, reducing training time.

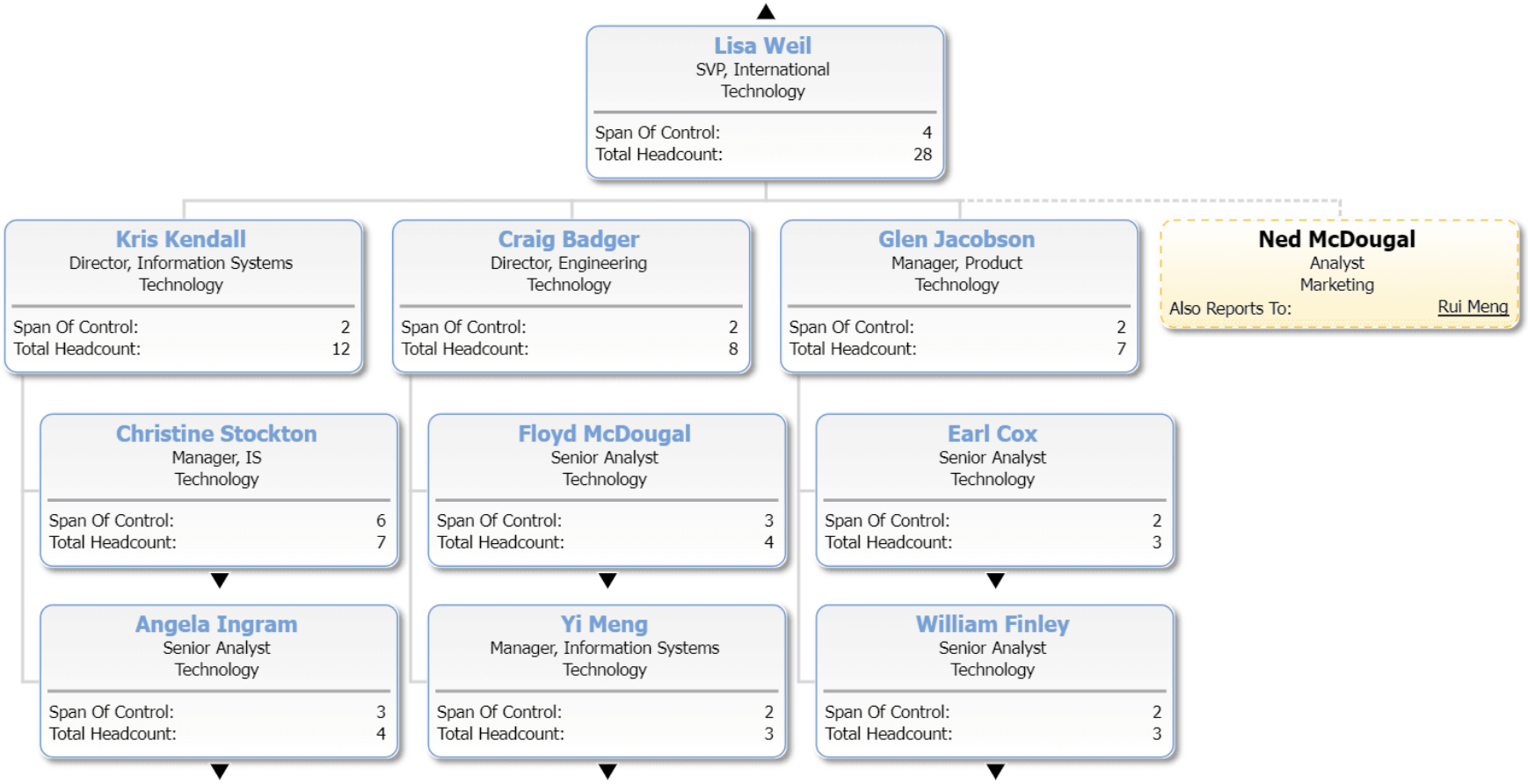
- **Disadvantages**

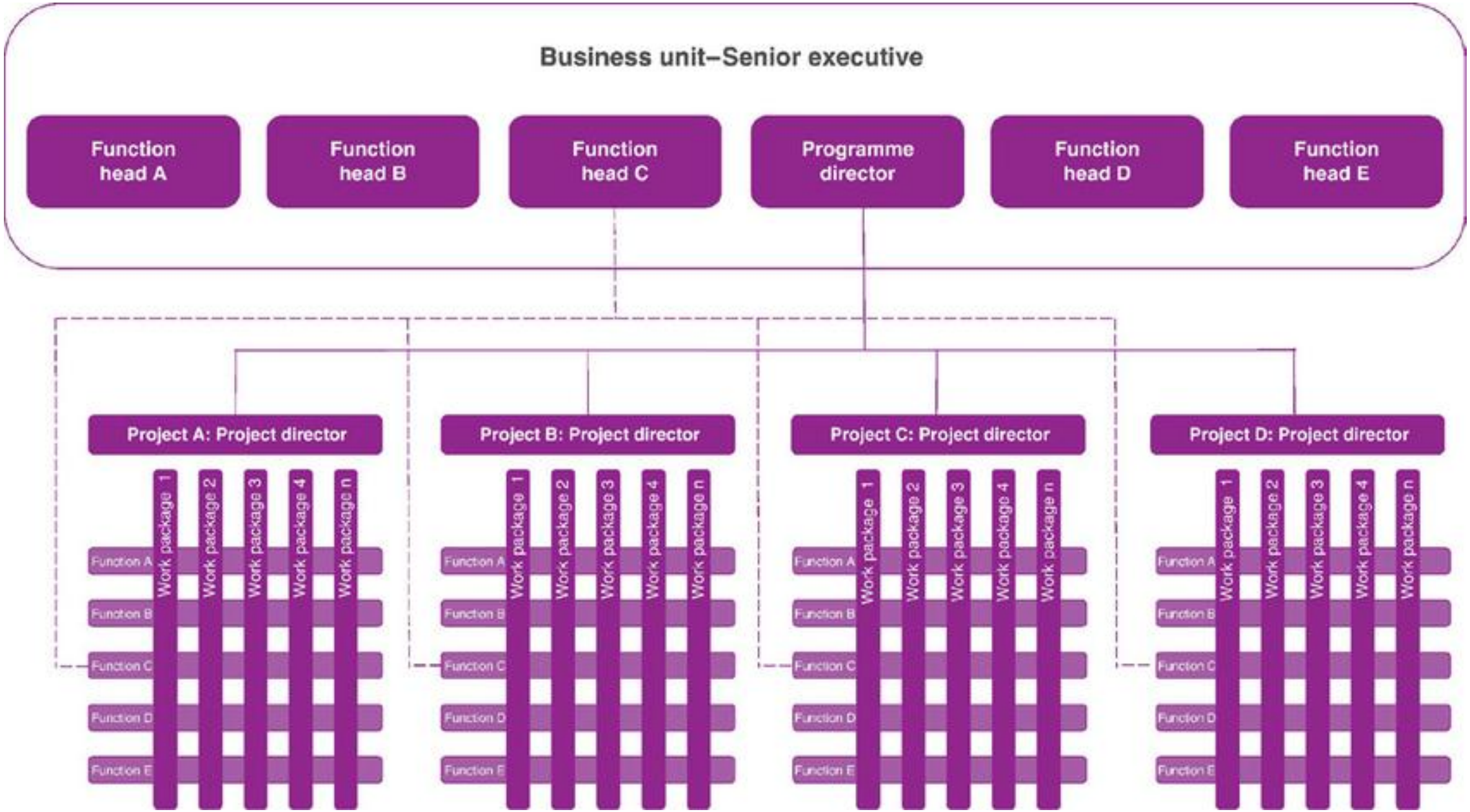
1. **Limited skill diversity**
    1. If the project needs multi-sector expertise, the unit may not have the needed specialists.
2. **Part-time project participation**
    1. Staff often have other program duties, creating scheduling conflicts.
3. **Slow progress on cross-sector tasks**
    1. The unit may struggle when the project involves other disciplines.

- **Example:**
- A **nutrition improvement project** under the Ministry of Health needs both nutritionists and civil engineers (for water system repair). Since the Health Unit does not have civil engineers, the project struggles to find the needed skill. Work gets delayed because authority lies only within the health sector.
- **Validation:**
- This proves that the programmatic structure works only for single-sector projects and becomes weak when cross-sector skill is required.

Matrix Based Organization

- A matrix structure allows staff to work under two directions:
  - Their **functional/program manager**
  - A **project manager** for specific projects
  - Staff may be assigned temporarily to projects depending on expertise.
- Example:
- A monitoring and evaluation (M&E) specialist reports to the M&E Unit but works part-time on four different NGO projects.
- **Characteristics**
  - People report to both functional units and project managers
  - Resources are shared across multiple projects
  - Allows a wide mix of expertise
  - Most flexible structure for development organizations


Lisa Weil
SVP, International Technology
Span Of Control: 4
Total Headcount: 28
Kris Kendall
Director, Information Systems Technology
Span Of Control: 2
Total Headcount: 12
Craig Badger
Director, Engineering Technology
Span Of Control: 2
Total Headcount: 8
Glen Jacobson
Manager, Product Technology
Span Of Control: 2
Total Headcount: 7
Ned McDougal
Analyst Marketing
Also Reports To: Rui Meng
Christine Stockton
Manager, IS Technology
Span Of Control: 6
Total Headcount: 7
Floyd McDougal
Senior Analyst Technology
Span Of Control: 3
Total Headcount: 4
Earl Cox
Senior Analyst Technology
Span Of Control: 2
Total Headcount: 3
Angela Ingram
Senior Analyst Technology
Span Of Control: 3
Total Headcount: 4
Yi Meng
Manager, Information Systems Technology
Span Of Control: 2
Total Headcount: 3
William Finley
Senior Analyst Technology
Span Of Control: 2
Total Headcount: 3


Business unit–Senior executive
Function head A
Function head B
Function head C
Programme director
Function head D
Function head E
Project A: Project director
Project B: Project director
Project C: Project director
Project D: Project director
Work package 1
Work package 2
Work package 3
Work package 4
Work package n
Function A
Function B
Function C
Function D
Function E

- **Advantages**

1. Efficient use of scarce experts

1. Example: One water engineer can support four rural water projects rather than sitting idle in one project.

2. Better knowledge sharing

1. Staff interact across departments, improving collaboration.

3. Supports professional development

1. Specialists maintain ties with their functional units and continue learning.

4. Useful in dynamic environments

1. When donor priorities change, staff can be reallocated quickly.

- **Disadvantages**

1. Complex reporting

1. Staff may receive instructions from two managers, which can cause confusion.

2. Competition for resources

1. Multiple projects may need the same expert at the same time.

3. Higher coordination effort

1. Project managers must negotiate for resources rather than directly controlling them.

4. Requires strong communication and time management

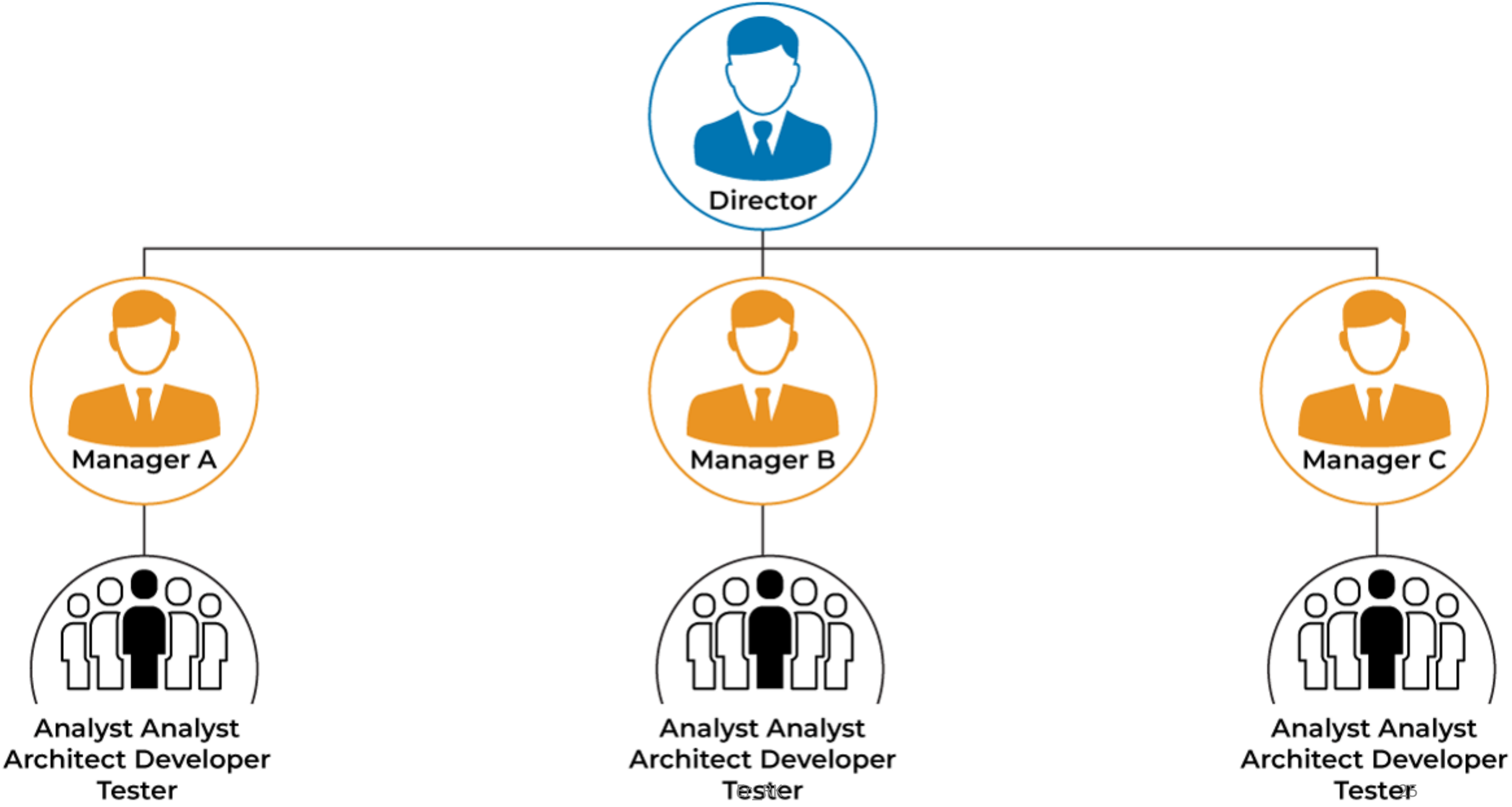
- **Example:**
- A national NGO running projects in **Education**, **Health**, and **WASH** shares specialists across all projects.
- A data analyst works on an education dashboard, a health tracking tool, and a water quality assessment
- Each project manager needs the analyst at the same time
- Coordination meetings are needed to decide priority
- **Validation:**
- This demonstrates that matrix structures make the best use of experts but require careful scheduling and cooperation.

Project Based Organization

- The project manager has **full authority** over all project resources. Team members are assigned **exclusively** to the project and report only to the project manager.
- This structure is common in **large, complex, or donor-funded projects**.
- Example:
- A **large hydropower project** in Nepal (such as Upper Tamakoshi or Arun III) forms a completely separate project unit with its own engineers, HR staff, procurement staff, and finance officers.

Projectized Organization Structure

Smartsheet Inc. © 2021


Director
Manager A
Manager B
Manager C
Analyst Analyst
Architect Developer
Tester
Analyst Analyst
Architect Developer
Tester
Analyst Analyst
Architect Developer
Tester

Characteristics

- Project manager has total control over planning, staffing, and decision-making
- Team members have strong project loyalty
- Communication is centralized
- Fast response to issues
- Little dependence on functional departments

- **Advantages**

1. Fast decision-making

1. The project manager approves resources immediately.

2. High team commitment

1. Staff work full-time on the project and understand goals deeply.

3. Clear accountability

1. No confusion about authority or responsibility.

4. Strong control over cost, schedule, and quality

- **Disadvantages**

1. High cost of staffing

1. Full-time teams mean more resources than needed during slow periods.

2. Duplication of roles

1. Example: Every large project may need its own finance officer or engineer, increasing overall cost.

3. Knowledge isolation

1. Lessons learned may not flow easily to other projects.

4. Uncertain career progression

1. When a project ends, staff may not know where they will be reassigned.

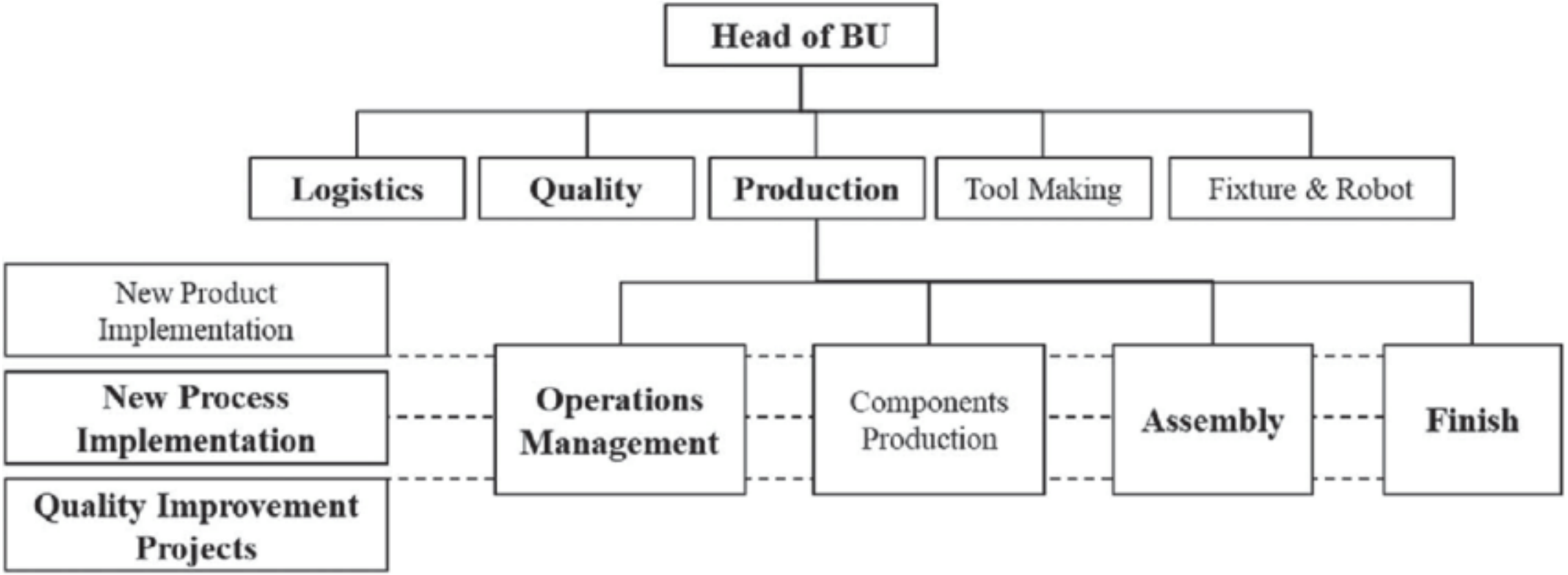
- **Example:**
- A donor-funded **Water Supply and Sanitation Project** forms a project-based unit.
- The project hires engineers, social mobilizers, accountants, and procurement specialists
- They work only on this project for 5 years
- When the project ends, contracts finish and staff must find new assignments
- **Validation:**
- This confirms both strengths (clear authority, fast decisions) and weaknesses (resource duplication, end-of-project job uncertainty).

Why Clear Responsibilities Matter

- **A. Reduces duplication of work**
  - Example: Two survey teams preparing the same map for a road project wastes money. A clear responsibility chart avoids this.
- **B. Improves accountability**
  - Example: If a delay occurs in a hydropower transmission line, the responsibility matrix helps identify whether the issue belongs to design, procurement, or civil works.
- **C. Supports quality control**
  - Example: In a software project for a Nepali college, quality assurance is assigned to a dedicated tester. This ensures independent checking rather than relying on programmers.
- **D. Helps manage risk**
  - Example: During monsoon, a construction project assigns a risk officer to monitor landslide-prone areas, preventing structural damage.

Line of Business (LOB) Organization

- The **Line of Business (LOB)** structure groups activities based on major product or service lines. Each line operates almost like a small company inside the larger organization.
- **Key Characteristics**
- Each line manages its own operations, budget, and strategic goals
- Decision-making is faster because authority stays within the unit
- LOB leaders are responsible for performance
- **Example**
- Telecom companies like **NTC** have separate LOB units such as GSM service, FTTH service, and Customer Care.
- Banks divide operations into Retail Banking, SME Banking, and Digital Banking.
- Each line has its own head who is accountable for growth, customer satisfaction, and operational improvement.


Head of BU
Logistics
Quality
Production
Tool Making
Fixture & Robot
New Product Implementation
New Process Implementation
Quality Improvement Projects
Operations Management
Components Production
Assembly
Finish

Core Idea of LOB Structure

- In a traditional vertical structure, the organization is arranged by functions such as HR, finance, or operations. In an LOB structure, the arrangement follows the **type of service or product** provided.
- Each line is responsible for:
  - Its own strategic direction
  - Its own financial performance
  - Development of specialized services
  - Managing customer experience
  - Coordinating projects that belong to that business line

Characteristics

- **A. Independent business units**
  - Each LOB operates almost like a mini-organization with its own manager, budget, staff, and performance measures.
- **B. Strong customer orientation**
  - LOBs focus on meeting the needs of a specific customer group or market.
- **C. Focused decision-making**
  - Managers of each line make quick decisions because authority is concentrated within that business line.
- **D. Alignment of resources**
  - Human resources, technology, and investments are grouped according to the line's goals.
- **E. Performance measured separately**
  - Each business line tracks its revenue, cost, and growth independently.

How LOB Works in Daily Operations

- A company may have several business lines. For example:
- Retail Banking
- SME Banking
- Digital Banking
- Remittance Services
- Each line maintains its own projects, staff, and operational priorities. They report performance to top management but operate with considerable autonomy.

Roles and Responsibilities Within an LOB

- **A. LOB Manager**
- Leads the entire line of business
- Sets targets and strategies
- Controls budget
- Approves major projects within the line
- Coordinates with top management
- **B. Project Managers under LOB**
- Handle projects linked to the business line
- Report to the LOB Manager
- Manage staff, vendors, deadlines, and outputs

C. Operational Staff

•Provide daily service delivery based on the line's specialty

•Engage directly with customers

•Maintain work quality standards

D. Support Units (Shared Across Organization)

•HR, finance, IT, and legal support the LOB as needed

•Provide resources, systems, and compliance guidance

Line of Business (LOB) Organization

- **LOB Manager:** Oversees operations, revenue, and performance
- **Project Manager (within LOB):** Leads project-based work tied to that business line
- **Operational Teams:** Handle day-to-day service delivery
- **Support Teams:** Provide HR, IT, procurement help

- **Advantages**
- Clear accountability
- Better focus on customer needs
- Faster decisions
- Encourages innovation within each business line

- **Challenges**
- Risk of duplication of resources
- Internal competition
- Need strong coordination

A. Telecom Sector Example: NTC or Ncell

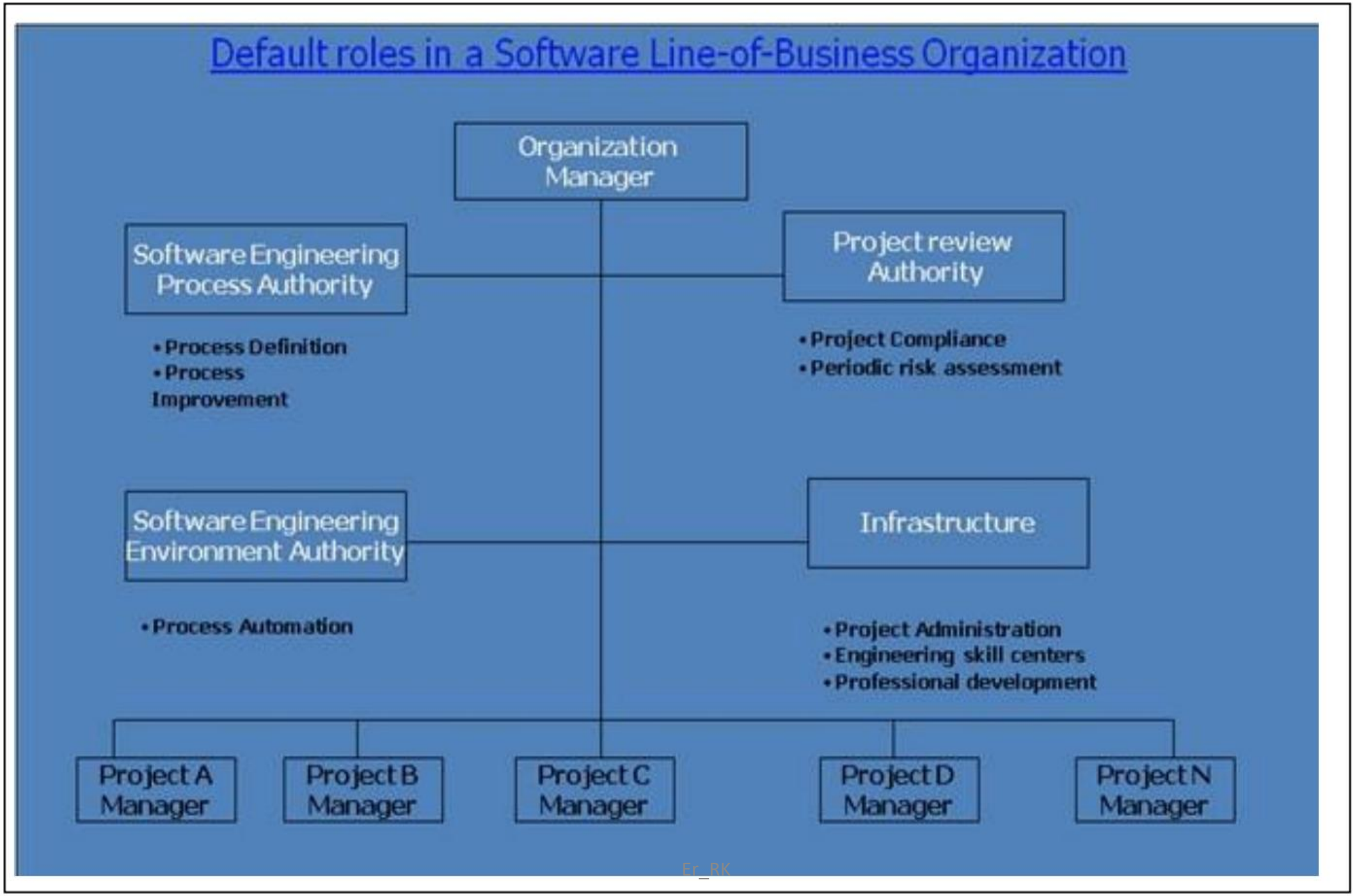
- They have multiple business lines:
  - GSM Mobile Service
  - Internet and FTTH Services
  - Enterprise Solutions
  - Prepaid and Postpaid Billing
- Each business line has:
  - Dedicated manager
  - Separate technical and marketing teams
  - Its own customer support processes
- **Validation:**
- If the GSM unit wants to introduce VoLTE, it runs its own project and budget plan without affecting FTTH or enterprise services.

LOB in Project Management Context

- When a project arises within a business line:
  - The **LOB manager** becomes the project sponsor
  - The project aligns with the LOB's strategic goals
  - A project manager is appointed for execution
  - Other LOB staff provide subject expertise
  - Support units (finance, IT, HR) provide shared services
- **Example**
- A Digital Banking LOB launches an online loan-application platform. The LOB manager approves the project, assigns internal IT staff, and coordinates with compliance and risk units. This shows how LOB integrates strategy with project work.

Main features of default organization

- Responsibility for process definition & maintenance is specific to a cohesive line of business.
- Responsibility for process automation is an organizational role & is equal in importance to the process definition role.
- Organizational role may be fulfilled by a single individual or several different teams.


Default roles in a Software Line-of-Business Organization
Organization Manager
Software Engineering Process Authority
• Process Definition
• Process Improvement
Project review Authority
• Project Compliance
• Periodic risk assessment
Software Engineering Environment Authority
• Process Automation
Infrastructure
• Project Administration
• Engineering skill centers
• Professional development
Project A Manager
Project B Manager
Project C Manager
Project D Manager
Project N Manager

- **1. Organization Manager**
  - This is the person who oversees the entire software business unit.
- **Simple Meaning**
  - This manager makes sure all projects, processes, tools, and teams work together smoothly.
- **Example**
  - In a Nepali software company building online banking systems, the **Head of Software Division** plays this role.
  - They ensure:
  - All projects follow the same standards
  - Teams get required tools and resources
  - Risks are monitored

- **2. Software Engineering Process Authority**
  - This group defines and improves how software is developed across all projects.
- **Simple Meaning**
  - They decide **how** the software should be built.

    They create standard procedures so that all teams follow the same coding and testing practices.
- **Key Tasks**
  - Define development processes
  - Improve workflows over time
- **Example**
- If the company decides that every project must use **Agile development**, this authority creates the rules:
  - How sprints should work
  - How code reviews should be done
  - How testing should be documented
  - This ensures Project A, B, C, and D follow the same process.

- **3. Project Review Authority**
  - This group regularly checks whether each project is following rules and progressing correctly.
- **Simple Meaning**
  - They act like an internal inspection team. They make sure projects are compliant and identify risks early.
- **Key Tasks**
  - Verify project progress
  - Check compliance with standards
  - Conduct periodic risk assessments
- **Example**
- A banking app project must follow strict security rules.
  If Project C misses encryption steps, this authority alerts the organization manager so corrections can be made early.

- **4. Software Engineering Environment Authority**
  - This group manages the tools and automation systems used by all project teams.
- **Simple Meaning**
  - They maintain the software tools needed for development, testing, and deployment.
- **Key Tasks**
  - Process automation
- Managing development tools (e.g., Git, CI/CD systems, testing frameworks)
- **Example**
- They install and support:
  - GitLab or GitHub
  - Jenkins for automation
  - Jira for task management
  - Testing tools
  - When Project A needs a new testing server, this team sets it up.

- **5. Infrastructure Authority**
  - This group handles supporting activities like:
  - Administrative work
  - Skill development
  - Training
  - Internal engineering resources
- **Simple Meaning**
  - They make sure the teams have the right people, skills, and administrative support.
- **Key Tasks**
  - Training employees
  - Managing engineering skill centers
  - Maintaining shared infrastructure
- **Example**
- If the company wants developers to learn cloud computing:
- This authority organizes AWS or Azure training
- It ensures all projects have cloud access and servers

- **6. Project Managers (Project A, B, C, D, N, etc.)**
  - These managers lead individual software projects.
- **Simple Meaning**
  - Each project manager handles one software product or client assignment.
- **Their Work Includes**
  - Planning work
  - Managing teams
  - Coordinating with all authorities mentioned above
- **Example**
- Project A Manager.
  - Handles the development of a **mobile wallet app**.
- Project B Manager:
  - Works on a **college management system** for Nepal.
- Project D Manager:
  - Leads a **government e-procurement portal**.
- All these managers receive support from the Process Authority, Review Authority, Environment Authority, and Infrastructure teams.

- **Software Engineering Process Authority (SEPA):**
  - The SEPA facilities the exchange of information & process guidance both to & from project practitioners.
  - This role is accountable to General Manager for maintaining a current assessment of the organization's process maturity & its plan for future improvement.
- **Project Review Authority (PRA**):
  - The PRA is the single individual responsible for ensuring that a software project complies with all organizational & business unit software policies, practices & standards A software Project
  - Manager is responsible for meeting the requirements of a contract or some other project compliance standard.
- **Software Engineering Environment Authority(SEEA**):
  - The SEEA is responsible for automating the organization's process, maintaining the organization's standard environment, Training projects to use the environment & maintaining organization-wide reusable assets The SEEA role is necessary to achieve a significant ROI for common process.
- **Infrastructure:**
  - An organization's infrastructure provides human resources support, project-independent research & development, & other capital software engineering assets.

Project Monitoring and Control

Chapter_6

What is EVM

- EVM is a set of measurements that compares:
  - **What you planned to do by today**
  - **What you actually completed by today**
  - **What you actually spent by today**
- It uses a baseline plan (scope + schedule + budget) and converts progress into money value.

Why EVM is used

- EVM is used because it gives early, objective signals about project health.
  - It detects schedule and cost problems early (often before people “feel” the project is failing).
  - It avoids false comfort, like “We used only 40% budget” when you completed only 20% of work.
  - It gives a disciplined way to forecast final cost and finish date.

When to use EVM

- Use EVM when the project has:
  - A defined scope and deliverables (WBS exists).
  - A planned schedule with milestones.
  - A budget broken down by work packages.
  - Enough duration and complexity that tracking matters.
- Typical use cases:
  - Software development projects with multiple modules and phases.
  - Construction and infrastructure projects.
  - Government and donor-funded projects with strict reporting.
  - Large campus systems such as EMIS, ERP, network deployment.
- Avoid or keep it lightweight when:
  - The project is very small or very short.
  - Scope changes daily without control.
  - You cannot measure progress reliably.

Where EVM fits in the project lifecycle

- EVM is mainly used in **Execution and Monitoring/Controlling**, but it depends on good planning.
  - **Initiation**: decide if EVM will be used.
  - **Planning**: build WBS, schedule, cost baseline (this is essential).
  - **Execution**: collect actual cost, measure progress.
  - **Monitoring/Control**: calculate EVM metrics, take corrective actions.
  - **Closing**: compare planned vs actual, record lessons learned.

How EVM works, step by step

- **Step 0: Create the baseline (this is mandatory)**
- You cannot do EVM without a baseline.
- You need:
  1. **WBS** (Work Breakdown Structure): break work into work packages.
  2. **Schedule**: planned start and finish for each work package.
  3. **Budget**: assign cost to each work package.
  4. **Measurement rule**: define how you will measure progress (0/100, 50/50, percent complete, milestone-based, etc.)
- The result is your **Performance Measurement Baseline (PMB)**.

Step 1: Define the three core values (PV, EV, AC)

- At every reporting date (weekly or biweekly), collect:
- **A) Planned Value (PV)**
- PV is the budgeted value of the work that should have been done by today.
- Think: “According to the plan, how much work should be completed by now, in money terms?”
- PV is also called **BCWS**.
- **B) Earned Value (EV)**
- EV is the budgeted value of the work that is actually completed by today.
- Think: “Based on actual progress, how much planned budget have we earned?”
- EV is also called **BCWP**.
- **C) Actual Cost (AC)**
- AC is what you have actually spent by today for the work done.
- AC is also called **ACWP**.

Step 2: Compute schedule and cost variances (SV, CV)

- These tell you if you are behind or ahead.
- **Schedule Variance (SV)**
  - SV = EV − PV
  - SV > 0: ahead of schedule
  - SV = 0: on schedule
  - SV < 0: behind schedule
- **Cost Variance (CV)**
  - CV = EV − AC
  - CV > 0: under budget
  - CV = 0: on budget
  - CV < 0: over budget

Step 3: Compute performance indices (SPI, CPI)

- Variances are in money units. Indices show performance ratios, which are easier to compare.
- **Schedule Performance Index (SPI)**
  - SPI = EV / PV
  - SPI > 1: ahead of schedule
  - SPI = 1: on schedule
  - SPI < 1: behind schedule
- **Cost Performance Index (CPI)**
  - CPI = EV / AC
  - CPI > 1: under budget (good)
  - CPI = 1: on budget
  - CPI < 1: over budget (bad)

Step 4: Forecast the final cost (EAC) and remaining work (ETC)

- Forecasting is one of the biggest reasons EVM is used.
- Let:
- **BAC** = Budget at Completion (total planned budget)
- Common formulas:
- **Estimate at Completion (EAC)**
- If current cost efficiency continues:
  - EAC = BAC / CPI
- If you expect remaining work to follow original plan (rare):
  - EAC = AC + (BAC − EV)
- **Estimate to Complete (ETC)**
- ETC = EAC − AC

Step 5: Interpret results and take control actions

- EVM numbers are not for decoration. They must lead to action, such as:
  - Re-allocating resources to delayed work packages
  - Removing blockers
  - Reducing scope through formal change request
  - Negotiating timeline changes
  - Improving productivity, quality, or procurement plans

6) A complete worked example (clear and realistic)

- **Scenario**
  - A web-based “Student Attendance and Class Log System” project has:
  - Total budget (BAC): **NPR 200,000**
  - Planned duration: 10 weeks
  - Today is end of week 5 (halfway)
- According to plan, you should complete **50%** of work by now.
- So:
  - PV = 50% of 200,000 = **NPR 100,000**
- Actual progress check shows only **40%** of scope is completed.
- So:
  - EV = 40% of 200,000 = **NPR 80,000**

- Accounting shows you spent **NPR 120,000** so far.
- So:
  - AC = **NPR 120,000**
- Now calculate:
- **Variances**
  - SV = EV − PV = 80,000 − 100,000 = **−20,000** (behind schedule)
  - CV = EV − AC = 80,000 − 120,000 = **−40,000** (over budget)
- **Indices**
  - SPI = EV / PV = 80,000 / 100,000 = **0.80** (slow progress)
  - CPI = EV / AC = 80,000 / 120,000 = **0.67** (poor cost efficiency)

- **Forecast final cost**
  - EAC = BAC / CPI = 200,000 / 0.67 ≈ **298,507** (approx.)
  - Meaning: if this continues, the project may finish around **NPR 300,000**, not 200,000.
  - ETC = EAC − AC ≈ 298,507 − 120,000 ≈ **178,507**
- **Interpretation in plain words**
  - At week 5:
  - You are behind schedule (40% done instead of 50%).
  - You are over budget (spent 120k, but earned only 80k value).
  - If nothing changes, you may need about 100k extra to finish.

7) How to measure EV correctly (the practical "how")

- The biggest challenge is measuring "percent complete" honestly.
- Better approaches:
- **A) Milestone-based earning (best for software)**
- Example for a module worth NPR 50,000:
  - UI complete: 20%
  - API complete: 30%
  - Testing complete: 30%
  - Deployment + documentation: 20%
- You only "earn" the portion that is truly finished.

- **B) 0/100 rule**
  - You earn 0% until the work package is complete, then earn 100%.
    Good for short tasks where partial credit is misleading.
- **C) 50/50 rule**
  - You earn 50% when work starts, and 50% when it finishes.
    Useful when tasks are medium-sized.
- **D) Percent complete (use carefully)**
  - Only if you have objective evidence (test cases passed, review accepted, etc.)

Advantages of EVM

- Combines scope, time, and cost in one system
- Early warning for schedule and budget problems
- Makes progress reporting more objective
- Supports forecasting (EAC, ETC)
- Helps management take corrective action with evidence
- Strong for reporting to stakeholders (donors, management, steering committee)

Disadvantages and limitations

- Needs strong planning (WBS, schedule, cost baseline)
- Requires accurate progress measurement, which is hard in software if poorly defined
- If scope changes frequently without change control, EVM becomes unreliable
- Can create “reporting pressure” and biased progress reporting if culture is weak
- More overhead for small projects

Monitoring Techniques and Tools

- Project monitoring means collecting reliable facts about progress, cost, quality, and risks, then comparing them with the plan so you can act early.
- It is not supervision or fault-finding.
- It is a control loop: plan, measure, compare, correct.

What it is

- **Project monitoring** is the continuous process of tracking:
  - Work progress against schedule (tasks, milestones, deliverables)
  - Cost against budget (planned vs actual spending)
  - Quality against standards (defects, rework, acceptance results)
  - Scope stability (change requests, requirement churn)
  - Risks and issues (what might happen, what is happening)
  - Resources (availability, utilization, productivity)
- The output of monitoring is decision-ready information, not raw data.

Why monitoring is needed

- Monitoring helps you:
  - Detect slippage early, before deadlines are missed
  - Control cost growth before it becomes unmanageable
  - Reduce rework by catching quality problems early
  - Keep stakeholders aligned with facts, not assumptions
  - Identify bottlenecks and unblock the team
  - Support fair reporting and accountability
- Without monitoring, a project often “looks fine” until it suddenly fails.

When to monitor

- Monitoring is not a one-time activity. It should be done:
  - Daily for critical operational signals (task flow, blockers)
  - Weekly for progress, cost, and issues (most common)
  - At every milestone or phase gate (formal review)
  - Whenever a major change request arrives (impact analysis)
  - After major incidents (defect spikes, vendor delay, staff change)
- For a 4–12 week student or department project, weekly monitoring is usually enough. For large projects, monitoring may be daily plus weekly governance.

Where monitoring is applied

- Monitoring is used across domains:
  - Software projects: sprints, releases, defect trends, deployment readiness
  - Construction: materials, labor productivity, inspection results
  - Network deployments: site readiness, device installation, connectivity tests
  - University systems (EMIS/ERP): module-wise progress, training status, data migration completion
- Monitoring also happens at different levels:
  - Team level (developer tasks, test cases)
  - Project manager level (schedule, budget, risks)
  - Sponsor level (milestones, benefits, major issues)

How monitoring works, step by step

- **Step 1: Define what you will measure (metrics)**
- You cannot monitor everything. Choose a small, useful set:
- **Schedule**
- Planned vs completed tasks
- Milestone completion rate
- SPI (if using EVM) or schedule variance
- **Cost**
- Actual spending vs budget
- CPI (if using EVM)
- Burn rate per week

Scope
•Number of change requests
•Requirement churn percentage
Quality
•Defects found per week
•Defect leakage (bugs found after release)
•Rework percentage
Risk and Issues
•Count of open issues, aging of issues
•High risks and mitigation status

- **Step 2: Collect data consistently**
- Use clear rules:
  - Progress is accepted only when it meets “done” criteria (tested, reviewed, documented, accepted)
  - Costs are collected from actual invoices, payroll estimates, or purchase records
  - Issues are logged with owner, deadline, and priority

- **Step 3: Compare with the baseline plan**
  - This is where monitoring becomes meaningful:
  - Are we ahead or behind schedule?
  - Are we over or under budget?
  - Is quality improving or getting worse?
  - Are risks increasing or decreasing?

- **Step 4: Analyze causes, not just symptoms**
- Examples:
  - “Coding is slow” is a symptom.
  - Root causes might be unclear requirements, unstable internet, new team members, or poor module integration.
- Use simple techniques:
  - 5 Whys
  - Fishbone (cause-effect)
  - Pareto (top causes)

- **Step 5: Take corrective or preventive actions**
- Monitoring without action is useless.
- Corrective actions:
  - Add resources to critical tasks
  - Remove blockers, clarify requirements
  - Re-plan a milestone
  - Improve testing and review discipline
- Preventive actions:
  - Extra training
  - Better checklists
  - Earlier code reviews
  - Vendor backup plan

- **Step 6: Communicate with the right format**
- Different audiences need different detail:
- Team: task-level, daily blockers
- Sponsor: milestone status, major risks, cost trend
- Client/user: deliverables, acceptance status, change requests

6) Monitoring techniques (methods) with examples

- **1. Status meetings (daily or weekly)**
- Purpose: quick alignment and escalation.
- Good practice:
  - Fixed agenda: progress, next steps, blockers, decisions needed
  - Time-boxed: 15 minutes daily or 30–60 minutes weekly
  - End with actions and owners
- Example (software project):
  - Monday review shows “Login module done” but testing blocked due to missing test accounts. Action: admin creates accounts by today.

- **2. Work performance reports**
- Purpose: formal, written record.
- Typical sections:
  - Completed work this period
  - Planned work next period
  - Variance (schedule/cost)
  - Risks/issues and decisions needed
- Example (university system implementation):
  - “Data migration is 70% complete, but department A is not providing verified data. Risk: inaccurate records. Decision needed: assign focal person.”

- **3. Milestone reviews and phase gates**
- Purpose: confirm readiness before moving to next stage.
- Common gates:
  - Requirements signed off
  - Design approved
  - UAT completed
  - Deployment authorized
- Example:
  - For EMIS module rollout, you do not start training until the module passes UAT with at least 95% critical test cases passed.

- **4. Inspection and quality audits**
- Purpose: ensure deliverables meet standards.
- Examples:
  - Code review audit: check if peer review happened for all merges
  - Document audit: verify version control, approvals, and completeness

- **5. Trend analysis**
- Purpose: understand direction over time, not just today’s status.
- Examples:
  - Defect trend: bugs per week should decrease near release
  - Velocity trend: sprint completion stability
  - Cost burn trend: spending rate vs plan

- **6. Earned Value tracking (if used)**
- Purpose: objective measurement of cost and schedule performance.
- Use when there is a baseline and measurable scope.

7) Tools used in monitoring

- **A) Scheduling and task tracking tools**
- Microsoft Project: baseline schedule, dependencies, critical path
- Jira: sprint boards, backlog, burndown, issue tracking
- Trello/Asana: lightweight task tracking for small projects
- Best for:
- Task progress, due dates, ownership, blockers

- **B) Dashboards and reporting tools**
- Power BI, Google Looker Studio, Excel dashboards
- Simple KPI dashboard in Sheets
- Best for:
- Sponsors and management who need a fast overview

- **C) Documentation and collaboration**
- Confluence, Google Drive, SharePoint
- Version control for docs (when needed)
- Best for:
- Meeting minutes, decisions, change logs, lessons learned

- **D) Version control and CI tools (software projects)**
- GitHub/GitLab, Bitbucket
- CI pipelines: GitHub Actions, GitLab CI, Jenkins
- Best for:
- Tracking work completion via pull requests, build status, test results

- **E) Communication tools**
  - Email, Teams, Slack, WhatsApp (for quick updates)
  - Use carefully: decisions must be recorded formally
- **F) Risk and issue tracking tools**
  - Issue log in Excel/Sheets
  - Jira issue types for risk and issue
  - Simple Kanban board: To Do, In Progress, Resolved
- Best for:
  - Clear ownership and follow-up, no missed problems

Advantages of strong monitoring

- Better control of time and cost
- Fewer surprises near deadlines
- Higher quality and less rework
- Better stakeholder trust and transparency
- Faster decision-making and escalation
- Better learning for future projects (lessons learned)

Disadvantages and common mistakes

- **Disadvantages**
  - Adds management overhead (time and documentation)
  - Can become “reporting for reporting’s sake” if poorly designed
  - Too many metrics can confuse everyone
- **Common mistakes**
  - Measuring activity instead of completion (hours worked instead of deliverables accepted)
  - Inflated progress reporting (claiming 90% done for weeks)
  - No clear “definition of done”
  - Issues tracked but not owned and not closed
  - Meetings without action items
  - Tools used, but data not updated regularly

Case: Campus network upgrade project (6 weeks)

- Plan:
  - Week 1–2: procurement and cabling
  - Week 3–4: switch configuration and installation
  - Week 5: testing and documentation
  - Week 6: training and handover
- Monitoring setup:
  - Weekly milestone review with checklist
  - Issue log (owner, priority, due date)
  - Gantt chart updated weekly
  - Testing checklist (connectivity, VLAN, throughput)

- Week 3 monitoring findings:
  - Cabling delayed due to vendor shortage.
  - Two buildings not ready (civil work incomplete).
    Actions:
  - Re-sequence: configure switches in lab first
  - Escalate: campus admin completes readiness by Friday
  - Adjust schedule baseline with approved change request
- Result:
  - Delay reduced from two weeks to five days because monitoring revealed the real blocker early.

Change Control Management,

- Change Control Management is the disciplined way of handling changes in a project so that useful changes are accepted and harmful or uncontrolled changes are avoided.
- In simple words, it answers this question: **What happens when someone asks to change the plan?**

What is Change Control Management

- Change Control Management is a formal process used to **identify, evaluate, approve or reject, and implement changes** to a project's scope, schedule, cost, quality, or resources.
- A change can be:
  - Adding new features
  - Modifying requirements
  - Changing deadlines
  - Adjusting budget
  - Replacing tools or technologies
  - Changing team members or vendors
- Change control ensures that every change is **understood before it is done**.

Why Change Control Management is necessary

- Projects rarely run exactly as planned. New needs appear, mistakes are discovered, and external conditions change. Without change control:
  - Scope grows silently (scope creep)
  - Costs increase without approval
  - Deadlines slip without explanation
  - Quality drops due to rushed work
  - Team morale suffers due to confusion
- Change control protects the project by making changes **visible, evaluated, and approved**.

When Change Control is used

- Change control is used **throughout the project**, mainly during execution and monitoring.
- Typical situations:
  - Client requests a new feature after development has started
  - Management wants faster delivery without extra resources
  - A regulation or policy changes mid-project
  - A technical solution proves unworkable
  - Budget cuts or extensions occur
- Important point:
  Not every suggestion is a change request. Only changes that affect **baseline elements** (scope, time, cost, quality) require formal change control.

Where Change Control applies

- Change control applies to:
  - **Scope baseline**: requirements, features, deliverables
  - **Schedule baseline**: task dates, milestones
  - **Cost baseline**: budget allocations
  - **Quality standards**: acceptance criteria
  - **Resources**: people, tools, vendors
  - **Documentation**: plans, designs, contracts
- In IT projects, change control is especially important because small changes can have large downstream effects.

Types of changes

- **Scope change**
  Adding, removing, or modifying features or requirements.
- **Schedule change**
  Changing deadlines, milestones, or task sequence.
- **Cost change**
  Increasing or decreasing budget.
- **Technical change**
  Switching technology, platform, or architecture.
- **Resource change**
  Changing team members, skills, or vendors.
- **Quality change**
  Modifying acceptance criteria or performance standards.

How Change Control works, step by step

- **Step 1: Change request submission**
- A change begins with a **formal change request**, not verbal instruction.
- A change request should include:
  - Description of the change
  - Reason for the change
  - Who requested it
  - Date and urgency
  - Affected areas (scope, time, cost, quality)
- Example:
  “Add SMS notification feature to student attendance system so parents receive daily updates.”

- **Step 2: Change logging**
- The change request is recorded in a **change log** so it is tracked and not forgotten.
- Typical change log fields:
- Change ID
- Description
- Requester
- Status (submitted, under review, approved, rejected)
- Decision date

Step 3: Impact analysis

- This is the most critical step.
- The project team analyzes:
  - **Schedule impact**: extra days or weeks required
  - **Cost impact**: additional budget needed
  - **Resource impact**: extra staff or skills required
  - **Quality impact**: risk of defects or rework
  - **Risk impact**: new risks introduced
- Example analysis:
  - SMS feature requires 2 extra weeks
  - Additional cost: NPR 25,000
  - Requires API integration and testing
  - Adds risk of message delivery failure

Step 4: Review and decision

- The change is reviewed by the appropriate authority:
  - Project Manager (small changes)
  - Change Control Board (medium to large projects)
  - Sponsor or client (contractual changes)
- Possible decisions:
  - Approved
  - Approved with conditions
  - Deferred
  - Rejected
  - The decision must be documented.

- **Step 5: Baseline update**
- If approved:
- Update scope documents
- Revise schedule and budget
- Update risk register
- Communicate changes to stakeholders
- This step is essential. A change without baseline update leads to confusion later.

- **Step 6: Change implementation**
  - The approved change is executed according to the updated plan.
  - Assign tasks
  - Track progress
  - Test and validate
  - Ensure quality standards are met
- **Step 7: Verification and closure**
  - After implementation:
  - Verify that the change achieved its objective
  - Obtain acceptance if required
  - Close the change request in the change log

Example Scenario: College EMIS implementation

- Original plan:
  - Modules: student record, attendance, exam result
  - Budget: NPR 500,000
  - Duration: 4 months
- Midway request:
  - Add online fee payment module
- Impact analysis shows:
  - Extra 3 weeks required
  - Additional cost: NPR 80,000
  - Requires payment gateway integration
  - Introduces security and compliance risks

- Decision:
  - Approved by management with extra budget and revised deadline
- Actions:
  - Schedule updated
  - Budget revised
  - Security testing added
  - Change documented and tracked
- Result:
  - System delivered with clear agreement, no disputes at handover

Advantages

- Prevents uncontrolled scope creep
- Maintains budget and schedule discipline
- Improves transparency and accountability
- Supports informed decision-making
- Reduces conflict with clients and stakeholders
- Improves project predictability and documentation

Disadvantages and limitations

- Adds paperwork and approval time
- Can slow down decision-making if overly bureaucratic
- May frustrate stakeholders who want quick changes
- Requires disciplined documentation and follow-up

- Poorly implemented change control can become an obstacle instead of support.

Common mistakes in Change Control

- Accepting changes verbally without documentation
- Skipping impact analysis
- Approving changes without adjusting budget or timeline
- Implementing changes before approval
- Not communicating approved changes to the team
- Treating every small suggestion as a major change

Change Control vs Scope Creep

- **Change Control**: planned, evaluated, approved, documented change
- **Scope Creep**: uncontrolled, informal, undocumented change
- Good projects allow change, but **only through control**.

Issue and Risk Tracking

- Issue and risk tracking is the practical system a project uses to make sure problems are not ignored, repeated, or forgotten.
- It is one of the most important parts of project control because most projects fail slowly through small, unmanaged issues and risks.

What is Issue Tracking

- An **issue** is a problem that has already happened and is affecting the project now.
- Examples of issues:
  - Developer cannot access the server (work stopped).
  - Vendor did not deliver switches on time (installation delayed).
  - Key staff member is absent (task ownership missing).
  - A module fails testing (rework needed).
- **Issue tracking** is the process of:
  - Logging the issue
  - Assigning an owner
  - Setting priority and due date
  - Monitoring progress
  - Closing it with a resolution

What is Risk Tracking

- A **risk** is a possible future event that can impact the project if it happens.
- Examples of risks:
  - Risk of internet outage during online examination week.
  - Risk that user training will be insufficient.
  - Risk that requirements will change after development starts.
  - Risk that data migration will be inaccurate.
- **Risk tracking** is the process of:
  - Identifying and recording risks
  - Estimating probability and impact
  - Planning responses (mitigation, avoidance, transfer, acceptance)
  - Monitoring risk triggers and response actions
  - Updating risk status over time

Why issue and risk tracking are necessary

- Without tracking:
  - Problems repeat because lessons are not recorded.
  - Small delays grow into major failure.
  - Stakeholders lose trust because surprises keep happening.
  - Team wastes time in confusion about who is solving what.
  - Decisions are taken late, when options are limited.
- With tracking:
  - Everyone knows what is open, what is urgent, and who owns it.
  - You can act early, not after damage is done.
  - Project status becomes fact-based.

When to track issues and risks

- **Issue tracking frequency**
  - Updated whenever a new issue is found
  - Reviewed daily (fast projects) or weekly (most projects)
- **Risk tracking frequency**
  - Updated weekly or at key milestones
  - Updated immediately when:
    - New risks appear
    - A risk becomes an issue
    - A major change request arrives

Where tracking is used

- Issue and risk tracking is used in:
  - Software projects (bugs, integration problems, testing failures)
  - Network projects (site readiness, device failure, cable issues)
  - Construction projects (materials shortage, weather delays)
  - University systems (EMIS/ERP adoption, training gaps, data accuracy)
- It is also used at different levels:
  - Team level: technical issues, task blockers
  - Project level: schedule, vendor, quality, scope risks
  - Sponsor level: budget and delivery risks, policy constraints

How to track issues, step by step

- **Step 1: Create an Issue Log**
- An issue log can be in Excel/Google Sheets, Jira, Trello, or MS Project.
- Minimum fields:
  - Issue ID
  - Date reported
  - Description
  - Category (schedule, cost, quality, vendor, technical, user)
  - Priority (High/Medium/Low)
  - Impact (what it affects)
  - Owner (responsible person)
  - Due date
  - Status (Open/In progress/Resolved/Closed)
  - Resolution notes

- **Step 2: Record the issue clearly**
  - A good issue description is specific.
  - Poor: “System is not working.”
    Better: “Attendance module fails to save records when internet disconnects, error code 504.”
- **Step 3: Prioritize and assign ownership**
  - Use simple priority rules:
  - High: stops work or impacts deadline
  - Medium: workaround exists, but harms quality or cost
  - Low: minor inconvenience, no major impact
  - Every issue must have one owner, not “team.”

- **Step 4: Decide the immediate action**
  - For each issue:
  - Fix now
  - Apply workaround
  - Escalate to sponsor or vendor
  - Defer with justification
- **Step 5: Review and close properly**
  - Do not close an issue until:
  - The solution is verified
  - The impact is removed
  - The resolution is documented

How to track risks, step by step

- **Step 1: Create a Risk Register**
- Minimum fields:
  - Risk ID
  - Risk statement (cause → event → impact)
  - Probability (Low/Medium/High or 1–5)
  - Impact (Low/Medium/High or 1–5)
  - Risk score (Probability × Impact)
  - Risk owner
  - Response strategy (avoid/mitigate/transfer/accept)
  - Mitigation actions
  - Triggers (warning signs)
  - Status (Open/Monitoring/Closed)
  - Notes

- **Step 2: Write the risk properly (cause → event → impact)**
- Example:
  "Due to limited training time (cause), staff may use EMIS incorrectly (event), leading to inaccurate student data and rework (impact)."
- This format prevents vague risks.
- **Step 3: Assess probability and impact**
- Simple scale:
  - Probability: 1 (rare) to 5 (very likely)
  - Impact: 1 (small) to 5 (severe)
  - Risk score = P × I
    Focus on the highest scores first.

Step 4: Choose a response strategy

- **Avoid**: change plan to remove the risk
  Example: avoid unstable hosting by using a reliable cloud provider.
- **Mitigate**: reduce probability or impact
  Example: early training, pilot testing, backup internet.
- **Transfer**: shift risk to a third party (contract, insurance)
  Example: vendor warranty, SLA with service provider.
- **Accept**: do nothing beyond monitoring
  Example: low impact risks.

- **Step 5: Define triggers and monitor**
- A trigger is an early warning sign.
- Example triggers:
- Training attendance below 70%
- Defect rate rising for two consecutive weeks
- Vendor delivery slipping by more than 3 days
- Requirements changed more than twice in one month

- **Step 6: Convert risks into issues when they happen**
- If a risk event occurs, it becomes an issue and must move into the issue log.
- Example:
  Risk: “Internet outage may interrupt online demo.”
  If outage happens on demo day, it becomes an issue: “Demo interrupted due to outage.”

- **For professional projects**
- Jira (best for software)
- Azure DevOps Boards
- GitHub Issues
- MS Project (basic issue log)
- Service desk tools (for IT operations)

Advantages

- Clear accountability (owner and due date)
- Faster problem resolution
- Fewer repeated mistakes
- Better stakeholder confidence
- Helps in audit and reporting
- Improves planning accuracy over time

Disadvantages and common mistakes

- **Disadvantages**
  - Requires discipline and time to update
  - Can become paperwork if not used for decisions
- **Common mistakes**
  - Logging issues without assigning an owner
  - No due dates, so issues never finish
  - Closing issues without verification
  - Treating risks as “fear list” without mitigation actions
  - Keeping risks hidden to “look good” in reports

Web-based Attendance and Teaching Log System for multiple college

- **Top risks**

1. Risk: Teachers may resist using the system
   Mitigation: short training + very simple UI + mobile-friendly page
   Trigger: fewer than 50% teachers log in during week 1
2. Risk: Data loss due to poor backup
   Mitigation: daily automated backup + restore testing
   Trigger: backup failure alert or missing backup file
3. Risk: Hosting downtime during peak hours
   Mitigation: reliable hosting + uptime monitoring
   Trigger: downtime more than 10 minutes

- **Current issues**

1. Issue: Attendance form is slow on mobile data
   Owner: frontend developer
   Due: Friday
   Action: optimize API calls, compress assets
2. Issue: Some colleges use different student ID formats
   Owner: data engineer
   Due: Wednesday
   Action: mapping table and validation rules

Software Configuration Management (SCM):

Software Configuration Management is a **disciplined and systematic process** used to control, organize, and monitor changes to software and its related documents throughout the Software Development Life Cycle, from initial requirement analysis to final deployment and maintenance.

In real projects, software is not a single file. It is a collection of many related items that continuously change. SCM ensures that these items remain **consistent, traceable, and approved** at every stage.

Software Artifacts Managed Under SCM

SCM covers all items that define the software system, including:

- **Source code**
  - Program files written in languages such as Java, Python, or C++.
- **Requirement documents**
  - Software Requirement Specification documents that describe what the system should do.
- **Design diagrams**
  - UML diagrams, architecture diagrams, and database designs.
- **Test cases**
  - Test plans, test scripts, and test reports.
- **Deployment files**
  - Configuration files, environment settings, and installation scripts.
- **User manuals**
  - End-user guides and technical documentation.

Why SCM Is Essential

Without SCM:

- Developers may overwrite each other's work.
- Old or incorrect versions may be deployed.
- Bugs may reappear due to unmanaged rollbacks.
- Accountability is lost.

SCM ensures that **the right version of the right product is delivered to the right user at the right time**, with full traceability.

Objectives:

1. Maintain Consistency of Software Products SCM ensures that all components of the software match each other. **Example** If version 2.0 of source code is released, the requirement document, test cases, and user manual must also correspond to version 2.0.	**2. Control Unauthorized Changes** SCM prevents changes that are not reviewed or approved. **Example** A junior developer cannot directly modify production code without approval from the project manager or technical lead.
3. Track Evolution of Software SCM records: • Who made a change • What was changed • When it was changed • Why it was changed This creates a complete history of the software. **Example** If a bug appears in the current version, developers can trace back to the exact commit where the issue was introduced.	**4. Support Teamwork and Parallel Development** Multiple developers can work on different features at the same time without conflict. **Example** One team member works on billing, another on reporting, and another on authentication, all using controlled versions.
5. Improve Software Quality and Reliability Controlled changes reduce errors, duplication, and unexpected failures. **Example** Only tested and approved modules are allowed into the final release, improving stability.	

SCM Activities:

1. Configuration Identification

Meaning	**Typical Configuration Items**
Configuration Identification is the process of **selecting and naming the items** that need to be controlled	• Source code files • Requirement documents • Design diagrams • Test plans

These items are called **Configuration Items (CIs)**.	• Release packages Each CI is given: • A unique name • A version number
Example In a payroll system: • `Payroll_Requirements_v1.0` • `Salary_Calculation_Module_v2.1` • `Payroll_TestCases_v1.3` This avoids confusion between similar files.	**Case Study: Academic Project** In a final-year project, students failed to label document versions properly. As a result, outdated diagrams were used during implementation, causing rework. After adopting configuration identification rules, confusion was eliminated.

2. Configuration Control

Meaning Configuration Control ensures that **no change is made without proper approval**. Changes are handled through a formal process.	**Change Control Steps** 1. Change request submitted 2. Impact analysis performed 3. Approval or rejection 4. Implementation 5. Documentation update
Example A request to add fingerprint login is raised. The project manager evaluates cost and time before approving the change. **Case Study: Hospital Management System** A hospital requested a last-minute change to prescription formatting. Without configuration control, developers directly modified the system, causing system downtime. After introducing approval procedures, changes were scheduled and tested properly.	

3. Configuration Status Accounting

Meaning	**Example**
Configuration Status Accounting involves **recording and reporting the current status** of configuration items. It answers: • Which version is current? • What changes are pending? • What has been approved?	A status report may show: • Module A: Version 1.2 (Approved) • Module B: Version 1.1 (Under Testing) • Module C: Version 1.0 (Deprecated)

Case Study: Software Company

During an audit, a company could not explain which version was deployed at a client site. After implementing status accounting, detailed version reports were maintained, improving transparency and trust.

4. Configuration Auditing

Meaning	**Types of Audits**
Configuration Auditing checks whether: • Approved procedures were followed • Documentation matches the delivered software • Standards are met	• **Functional audit** ○ Verifies that the software meets requirements. • **Physical audit** • Ensures correct versions of files are delivered.

Example

Before release, auditors verify that the code version matches the approved requirement document and test results.

Case Study: Banking Software

A bank performs strict configuration audits before every release to ensure compliance with security and legal standards. Any mismatch leads to rejection.

Comprehensive Case Study: Hospital Management System

Problem Without SCM	SCM Implementation	Outcome
• Multiple versions of prescription modules • Old modules deployed by mistake • No record of changes • Increased medical risk	• Configuration items identified • Git used for version control • Change requests approved by a committee • Status reports maintained • Audits conducted before deployment	• Only approved prescription modules deployed • Reduced medical errors • Improved accountability • Higher system reliability

Version Control Systems (VCS):

A Version Control System is a software mechanism that **records, manages, and preserves changes** made to files over time. It allows developers to view earlier versions, compare changes, and safely coordinate work among multiple contributors.

In software projects, files change continuously. Without version control, teams face overwritten work, missing updates, and confusion about which file is correct. A VCS creates a **single source of truth** with a complete history.

Why Version Control Is Essential in Software Projects

1. **Multiple contributors** work on the same files
2. **Frequent updates** are required due to bugs and new requirements
3. **Rollback** is needed when errors are introduced
4. **Accountability** is required for audits and maintenance

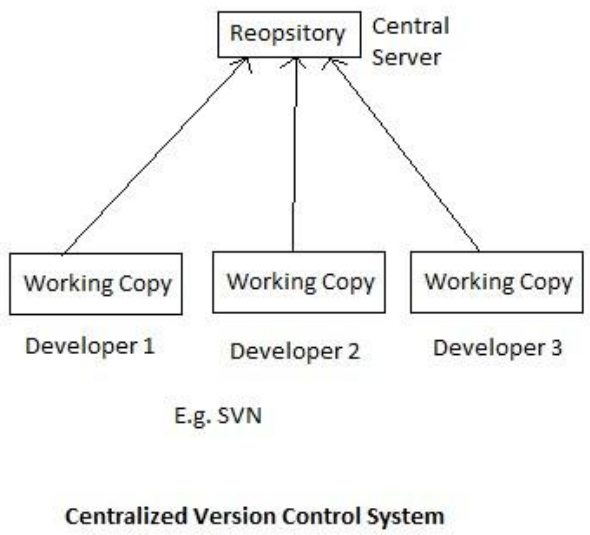
Version control ensures **control without blocking progress**.

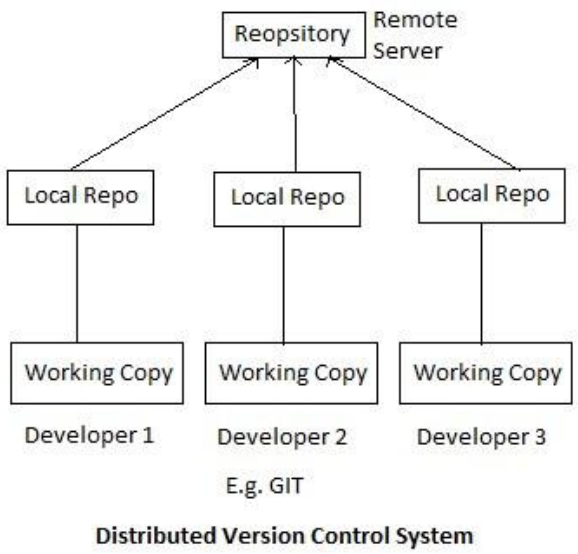
Core Concepts in Version Control

Repository	Commit
A repository is a storage location that contains: • Project files • Version history	A commit is a **snapshot of changes** saved into the repository.

• Metadata about changes Repositories may be local, centralized, or distributed.	Each commit usually includes: • What changed • Who changed it • When it was changed • Why it was changed
Branch A branch is a separate line of development that allows work to proceed independently.	**Merge** Merging combines changes from one branch into another.
Revert / Rollback Restores files to an earlier, stable version.	

Types of Version Control Systems





7.2.1 Centralized Version Control System (CVCS)

Concept In a centralized system, there is **one central repository** stored on a server. All developers connect to this server to: • Download files • Upload changes	**Characteristics** • Single point of control • Simple structure • Easy to manage access rights

If the server is unavailable, work stops.	
Characteristics • Single point of control • Simple structure • Easy to manage access rights	**Advantages** • Centralized administration • Clear control over codebase • Suitable for small, controlled teams
Limitations • Server failure halts development • Limited offline work • Slower for large teams	**Example** A small government IT department uses a centralized repository hosted on an internal server. Developers must be connected to the office network to commit changes.

Case Study: Government Payroll System

Scenario
A payroll application is developed for a public office. All code must remain within government infrastructure for security reasons.

Why Centralized VCS Works • Code never leaves the server • Access is strictly controlled • All changes are reviewed centrally	**Outcome** Security is maintained, but developers face delays during network outages.

7.2.2 Distributed Version Control System (DVCS)

Concept In a distributed system, **every developer has a full copy** of the repository, including complete history. Developers can work independently and synchronize changes later.	**Characteristics** • No single point of failure • Full offline capability • Strong support for branching and merging
Advantages • Faster operations • High reliability • Better collaboration • Encourages experimentation	**Limitations** • Slightly higher learning curve • Requires disciplined workflow

Example

A university software lab project uses a distributed system. Students work from home, make commits locally, and synchronize when internet access is available.

Detailed Working of Git (DVCS)

Repository Structure	Typical Git Workflow
• **Local repository**: Developer's personal copy • **Remote repository**: Shared central location for synchronization	1. Clone the repository 2. Create a branch 3. Make changes 4. Commit locally 5. Merge and push to the shared repository

Example

A developer adds a login feature in a separate branch. After testing, the branch is merged into the main branch without affecting other features.

Case Study: E-Commerce Website Development

Scenario	Problems Without Version Control
An e-commerce website is being developed by a team of eight developers.	• Files overwritten • Conflicting updates • Lost code
Solution Using Git	**Outcome**
• Each feature developed in separate branches • Code reviewed before merging • Bugs traced to specific commits	• Faster development • Fewer integration errors • Clear accountability

Version Control in Academic Projects

Example: Final Year Project

Without VCS	With VCS
• Files shared via pen drive or email • Confusion over latest version • Accidental deletion	• Clear history of contributions • Easy rollback • Fair evaluation of student work

Case Study: University Management System

Scenario
A team develops modules for attendance, grading, and finance.

Implementation	Outcome
• Separate branches for each module • Weekly merges • Tagged releases	• Smooth integration • Minimal conflicts • Timely submission

Role of Version Control in Maintenance

Version control is not limited to development. It is critical during:

- Bug fixing
- Feature enhancement
- Security patching

Example

A bug appears after deployment. Developers trace the issue to a specific commit and revert only that change instead of rewriting the entire code.

Centralized Version Control System: SVN

Meaning of SVN (Subversion)

SVN is a **Centralized Version Control System** where:

- A single central repository exists on a server

- All developers check out files from this server
- All commits are sent back to the same server

If the server is unavailable, work is disrupted.

Working of SVN	**Characteristics of SVN**
1. Developer checks out files from the server 2. Makes changes locally 3. Commits changes back to the server 4. Other developers update their copies	• Central server controls the codebase • Requires network connectivity • Linear version history • Simple branching and merging
Advantages of SVN	**Limitations of SVN**
• Centralized control and security • Easy access management • Simple concept for beginners • Suitable for regulated environments	• Single point of failure • Limited offline work • Slower for large teams • Branching is less flexible

Example of SVN Use

A government office maintains a centralized server where all payroll software code is stored. Developers can access and modify code only within office hours and network.

Case Study: Government Payroll System Using SVN

Context
A payroll system for civil servants must comply with strict data security rules.

Why SVN Was Chosen	**Outcome**
• Code must remain inside government servers • Access must be centrally controlled • Audit trail required	• Strong control and security • Slower development during network outages • Clear accountability during audits

Distributed Version Control System: Git

Meaning of Git

Git is a **Distributed Version Control System** where:

- Every developer has a complete copy of the repository
- Full history is stored locally
- Developers can work offline
- Changes are synchronized later

Git is the most widely used version control system in the software industry.

Working of Git	**Key Features of Git**
1. Clone the repository 2. Create a branch 3. Modify files 4. Commit changes locally 5. Merge branches 6. Push changes to shared repository	• Fast performance • Powerful branching and merging • Offline commits • Strong community support
Advantages of Git	**Limitations of Git**
• No single point of failure • High speed • Encourages experimentation • Ideal for Agile and DevOps	• Learning curve for beginners • Requires disciplined workflow • Poor practices can cause merge conflicts

Example of Git Use

A team developing a mobile application uses Git. One developer works on login features, another on payment processing. Both work independently and merge changes after testing.

Case Study: E-Commerce Website Development Using Git

Context
An e-commerce website is built by a distributed team working from different locations.

Problems Before Git	**Git-Based Solution**	**Outcome**
• Code conflicts • Delayed integration	• Feature-based branches	• Faster development • Reduced bugs

• Lost changes	• Code reviews before merge • Tagged releases	• Clear version history

Configuration Management Plan (CMP)

A **Configuration Management Plan (CMP)** is a **formal, documented guideline** that defines **how Software Configuration Management will be implemented and controlled** throughout a software project.

In simple terms, CMP answers this question:

How will we identify, control, track, and audit changes to all software-related items during the project?

The CMP acts as a **rulebook** for managing versions, changes, responsibilities, and tools.

Why it Is Required?

Software projects involve:	Without a CMP:
• Multiple developers • Frequent changes • Many documents and files • Long development timelines	• Different versions get mixed • Unauthorized changes occur • Responsibility becomes unclear • Releases become unstable

A CMP brings **discipline, predictability, and traceability**.

Objectives:

1. **Define configuration items clearly**
2. **Establish change control procedures**
3. **Assign roles and responsibilities**
4. **Standardize versioning and naming rules**
5. **Ensure auditability and compliance**
6. **Reduce errors caused by uncontrolled changes**

When CMP Is Prepared	Scope of CMP
• Prepared during the **project planning phase** • Approved before development starts	The CMP applies to: • Entire project life cycle

• Updated if project scope or tools change	• Development, testing, deployment, and maintenance • All software and documentation artifacts

Major Components of a Configuration Management Plan

1. Introduction and Purpose	Example
Description This section explains: • Why the CMP exists • What project it applies to • What it aims to achieve	"This Configuration Management Plan defines the procedures for identifying, controlling, and auditing configuration items for the Hospital Management System project."

2. Configuration Items (CI) Identification

Meaning	Types of Configuration Items	Each CI must have:
Configuration Items are **any artifacts that must be controlled** because changes to them affect the project.	• Source code files • Requirement documents • Design documents • Test cases and reports • Deployment scripts • User manuals	• Unique name • Version number • Owner

Example

Configuration Item	Type	Version
HMS_Requirements	Document	v1.0
Patient_Module	Source Code	v2.3
Test_Report	Test Document	v1.2

Case Study: Academic Project Failure

Students did not define configuration items. Multiple versions of reports and code were submitted, leading to confusion and poor evaluation. Introducing CI identification solved the problem.

3. Version Control Strategy

Purpose	Versioning Rules	Example
Defines how versions will be created, named, and managed.	• Major version change for major features • Minor version change for small enhancements • Patch version for bug fixes	• Version 1.0: Initial release • Version 1.1: Feature enhancement • Version 1.1.1: Bug fix

Example in Practice

Only stable versions are tagged as release versions. Experimental changes remain in development branches.

4. Change Control Process

Meaning	Change Request Process	Change Request Form Typically Includes
Change control ensures **no change is made without proper review and approval**.	1. Change request submitted 2. Impact analysis performed 3. Approval or rejection 4. Implementation 5. Documentation update	• Change description • Reason for change • Impact on cost and schedule • Priority level

Case Study: Hospital Management System

A request to modify prescription format was reviewed for medical safety, testing effort, and training impact before approval. This prevented unsafe deployment.

5. Roles and Responsibilities

Key Roles in CMP	Example
• **Project Manager** Approves changes and releases • **Configuration Manager** Maintains repositories and records • **Developers** Implement approved changes • **Quality Assurance Team** Verifies correctness and compliance	Only the configuration manager is allowed to merge changes into the main branch.

6. Configuration Status Accounting

Meaning	Example Status Report
Configuration Status Accounting records and reports: • Current version of each CI • Approved and pending changes • Release history	

Item	Version	Status
Login Module	v1.2	Approved
Payment Module	v1.3	Under Testing
Reports Module	v1.1	Deprecated

Case Study

A software firm failed an audit because it could not show which version was delivered. After implementing status accounting, audits were cleared successfully.

7. Configuration Auditing

Meaning	Types of Audits	Example
Configuration Auditing ensures: • Changes follow approved procedures • Delivered software matches documentation	• **Functional Audit** Checks whether requirements are satisfied • **Physical Audit** Verifies correct versions are included in the release	Before release, auditors verify that source code version matches approved requirements and test reports.

• Standards are followed		

8. Tools and Technologies

Common Tools Mentioned in CMP	Example
• Version control tools like Git or SVN • Issue tracking tools • Build and deployment tools	"Git will be used for source code management. JIRA will be used for change request tracking."

9. Access Control and Security

Purpose	Example
Defines who can: • View files • Modify files • Approve changes	• Developers can modify code • Test team can view code • Only project manager can approve releases

10. Release Management Interface

Description

Defines how SCM interacts with release management:

- Which versions can be released
- Release approval authority
- Release documentation

Comprehensive Case Study: Banking Software Project

Without CMP	With CMP
• Multiple versions deployed • Unauthorized changes • Failed audits	• Clear configuration items • Approved change control • Audited releases • Legal compliance

Advantages of Using CMP	Limitations of CMP
• Improved project control • Reduced rework • Clear accountability • Better quality • Audit readiness	• Requires documentation effort • Needs discipline and training • May seem slow for very small projects

Change Management

Change Management is a **formal and systematic process** used to **identify, evaluate, approve, implement, and document changes** in a software project.

In software projects, change is unavoidable. Requirements evolve, users request new features, bugs are discovered, and regulations change. Change Management ensures that such changes are **handled in a controlled way**, without harming cost, schedule, quality, or stability.

In simple terms:

Change Management decides **what can be changed, when it can be changed, and how it will be changed**.

Why Change Management Is Necessary?

Without proper change management:

- Project scope keeps expanding without control
- Cost and schedule overrun
- Quality degrades
- Team confusion increases
- Clients lose trust

Change Management protects the project from **chaos caused by uncontrolled modifications**.

Sources of Change in Software Projects	Objectives of Change Management
Changes may arise from: 1. **Client or user requests** New features, UI changes, reports, language support	• Control project scope • Minimize negative impact of changes • Ensure informed decision-making • Maintain documentation accuracy

2. **Technical issues** Performance problems, security vulnerabilities 3. **Bug fixes** Errors discovered during testing or after deployment 4. **Business or policy changes** New laws, compliance requirements, organizational rules 5. **Technology changes** Framework updates, database migration	• Improve customer satisfaction

Types of Changes

1. Functional Changes Changes to system features or behavior. **Example** Adding OTP-based login instead of simple password login.	**2. Non-Functional Changes** Changes related to performance, security, usability, or reliability. **Example** Improving response time of a search module.	**3. Corrective Changes** Fixing defects or errors. **Example** Fixing incorrect tax calculation logic.
4. Adaptive Changes Adjusting software to new environments or platforms. **Example** Updating software to support a new operating system.	**5. Preventive Changes** Changes made to avoid future problems. **Example** Refactoring code to reduce future maintenance issues.	

Change Management Process (Step-by-Step)

Step 1: Change Request Initiation A **Change Request (CR)** is formally submitted.	**Step 2: Change Impact Analysis** The proposed change is analyzed for its impact on:

A change request usually includes:

- Description of the change
- Reason for change
- Priority
- Requested by
- Expected benefits

Example

A hospital requests adding blood group information to patient reports.

- Cost
- Schedule
- Resources
- Risk
- Quality

This step answers:

Is the change feasible and worth implementing?

Example

Adding blood group data requires database changes, UI updates, testing, and staff training.

Step 3: Change Control Board (CCB) Review

A **Change Control Board** reviews the change request.

Members of CCB

- Project manager
- Technical lead
- Quality assurance representative
- Client or user representative

The CCB may:

- Approve the change
- Reject the change
- Defer the change

Example

The CCB approves the change but schedules it for the next release due to workload.

Step 4: Change Approval or Rejection

The decision is documented:

- Approved changes move forward
- Rejected changes are recorded with reasons

Example

A request to redesign the entire system UI late in the project is rejected due to high cost and delay.

Step 5: Change Implementation

Approved changes are:

- Assigned to developers

Step 6: Testing and Validation

Changes are tested to ensure:

- No new defects are introduced

• Implemented following coding standards • Integrated with existing system **Example** Developers update database schema and modify report templates.	• Existing functionality still works • Change meets requirements **Example** Regression testing is performed to ensure patient reports still generate correctly.
Step 7: Documentation Update All related documents are updated: • Requirements document • Design documents • User manuals • Release notes **Example** User manual is updated to explain the new blood group field.	**Step 8: Change Closure** The change is formally closed after: • Successful testing • Documentation update • Stakeholder confirmation

Case Study 1: Hospital Management System

Situation Without Change Management	Situation With Change Management	Outcome
• Doctors requested report changes verbally • Developers made direct code changes • Old reports stopped working • Patient safety risk increased	• Formal change requests submitted • Medical and technical impact analyzed • Approved changes tested thoroughly	• Safer system • Fewer errors • Improved trust from hospital staff

Case Study 2: Academic ERP System

Problem	Change Management Applied	Outcome
Students requested last-minute grade calculation changes.	• Change request logged • Impact on exams and deadlines analyzed • Approved for next semester release	• Current exams completed on time • New feature delivered later without disruption

Relationship Between Change Management and SCM

Change Management works **within Software Configuration Management**:

- SCM controls versions
- Change Management controls decisions
- CMP defines procedures
- Version control tools implement approved changes

Advantages of Change Management	Limitations of Change Management
• Prevents scope creep • Improves project predictability • Enhances quality • Maintains stakeholder confidence • Ensures audit readiness	• Requires documentation effort • Can slow rapid changes • Needs disciplined team behavior

Release Management

Release Management is the **process of planning, scheduling, controlling, and delivering approved changes** to users in the form of a software release.

While **Change Management decides what should change**, **Release Management decides when and how those changes are delivered** safely and reliably.

In simple words:

Release Management ensures that only tested, approved, and complete software versions are deployed to users.

Why Release Management Is Necessary

Without proper release management:

- Unstable software reaches users
- Frequent unplanned deployments occur
- Bugs reappear
- Users lose confidence
- Business operations are disrupted

Release Management ensures **stability, predictability, and quality** in software delivery.

Relationship with SCM and Change Management

- **SCM** controls versions and configuration items
- **Change Management** approves or rejects changes
- **Release Management** bundles approved changes and delivers them

All three work together to ensure controlled software evolution.

Objectives of Release Management

- Deliver stable and reliable software
- Minimize deployment risks
- Coordinate development, testing, and operations
- Ensure traceability of released versions
- Maintain release documentation

Key Concepts in Release Management

Release	**Baseline**	**Deployment**
A release is a **specific version of software** made available to users.	A baseline is a **stable reference version** from which further changes are made.	Deployment is the process of installing the release into the target environment.

Types of Software Releases

1. Major Release	2. Minor Release
Description Introduces significant new features or major changes. **Example** Version 3.0 of a university ERP system introducing online admission and payment modules.	**Description** Adds small enhancements or improvements without major changes. **Example** Version 2.1 adding new report formats.
3. Patch Release	**4. Emergency Release**
Description Fixes bugs or security issues. **Example** Version 2.1.1 fixing login errors.	**Description** A quick release to fix critical issues. **Example** Immediate security patch after vulnerability detection.

Release Management Process (Step-by-Step)

Step 1: Release Planning	Step 2: Release Build and Assembly
Description Defines: • What changes will be included • Release objectives • Target users • Release date Only **approved changes** from Change Management are considered. **Example**	**Description** Approved components are: • Integrated • Compiled • Packaged This forms a **release candidate**. **Example**

A hospital system plans a quarterly release including prescription updates and report enhancements.	Multiple modules developed separately are combined into a single deployment package.
Step 3: Testing and Validation **Types of Testing** • Unit testing • Integration testing • System testing • User Acceptance Testing Testing ensures that the release: • Meets requirements • Does not break existing functionality **Example** Before releasing a payroll system update, salary calculations are tested for all employee categories.	**Step 4: Release Approval** **Description** Final approval is given by: • Project manager • Quality assurance team • Client representative Only after approval can deployment proceed. **Example** A release is delayed by one week due to failed security testing.
Step 5: Deployment **Description** The release is deployed to: • Production environment • Client systems Deployment may be: • Manual • Automated **Example** An academic ERP system is deployed during semester break to avoid disruption.	**Step 6: Release Documentation** **Documentation Includes** • Release notes • Version number • New features • Fixed issues • Known limitations **Example** Release notes explain new grading features and known browser limitations.
Step 7: Post-Release Review	

Description After deployment: • System performance is monitored • User feedback is collected • Issues are logged **Example** User complaints after release help identify minor UI issues for the next update.

Case Study 1: Hospital Management System

Problem Without Release Management	**Release Management Introduced**	**Outcome**
• Developers deployed changes directly • Unstable versions used by doctors • Incorrect patient reports generated	• Quarterly planned releases • Mandatory testing • Approval before deployment	• Improved system reliability • Reduced medical risk • Increased user trust

Case Study 2: Banking Software

Context	**Release Management Approach**	**Outcome**
Banking software requires strict compliance and stability.	• Few releases per year • Extensive testing and audits • Rollback plans prepared	• Zero data loss • Regulatory compliance • High customer confidence

Role of Version Control in Release Management

Version control supports release management by:

- Tagging release versions
- Creating baselines

- Tracking changes included in a release

Example

Version 2.0 is tagged as a release point. Future changes start from this baseline.

Advantages of Release Management	Limitations of Release Management
• Predictable software delivery • Reduced deployment risk • Better coordination • Improved software quality • Clear accountability	• Requires planning effort • Slower delivery if poorly managed • Needs coordination across teams

Despite limitations, release management is essential for professional software development.

Overview of Project Management Tools

MS Project, JIRA, Trello

Project management tools help a team **plan work, assign responsibilities, track progress, manage risks/issues, and report status**. In software projects, these tools also support Agile boards, sprint planning, change tracking, and release visibility.

Microsoft Project (MS Project)

What it is used for?

MS Project is mainly used for **schedule-based project planning**. It is strong for traditional project management where you need clear timelines and task dependencies.

Key features

1. **Work Breakdown Structure (WBS) and task lists**
   Break the whole project into tasks and subtasks.
2. **Gantt Chart (timeline view)**
   Shows tasks on a timeline with start and finish dates.
3. **Task dependencies**
   Defines relationships like:
   - Finish-to-Start (Task B starts after Task A finishes)
   - Start-to-Start (both can start together)

4. **Resource assignment and workload**
   Assign people to tasks and check over-allocation.
5. **Milestones and critical path**
   Helps identify tasks that control the project end date.
6. **Baseline vs actual tracking**
   Set a baseline plan and compare it with actual progress.

Example (classroom-friendly)

Project: "Online Library System"

- Task: Requirement analysis (5 days)
- Task: UI design (7 days) depends on requirement analysis
- Task: Development (20 days) depends on UI design
  MS Project helps students visualize how delays in requirements affect the whole schedule.

Mini Case Study

Scenario: A campus ERP project has frequent delays.
Use of MS Project: The manager sets baseline dates and monitors variance weekly. The critical path shows testing is the main bottleneck, so extra testers are assigned.
Outcome: Delays reduce because the team focuses on schedule-critical tasks instead of random tasks.

Strengths	Limitations
• Best for schedules, dependencies, and timelines • Strong reporting for time management • Good for large, structured projects	• Not ideal for daily Agile task flow • Can feel heavy for small student teams • Needs careful updating to stay accurate

JIRA

What it is used for?

JIRA is widely used in software teams for **issue tracking and Agile project management**. It is excellent for managing:

- user stories
- tasks
- bugs
- sprints
- releases

Key features

1. **Backlog and sprint planning**
   - Create user stories
   - Prioritize backlog
   - Move items into a sprint
2. **Scrum and Kanban boards**
   Visual workflow: To Do → In Progress → Done
3. **Issue types**
   - Story (new feature)
   - Bug (defect)
   - Task (work item)
   - Epic (big feature group)
4. **Workflow and approvals**
   Custom steps like:
   - To Do → In Progress → Code Review → Testing → Done
5. **Reports and dashboards**
   - Burndown chart (sprint progress)
   - Velocity chart (team capacity)
   - Issue aging (delayed tasks)
6. **Release and version tracking**
   Mark issues that will go into Release 1.1, 1.2, etc.

Example

In a "Food Delivery App" project:

- Story: "User can register with email and phone"
- Bug: "OTP not received sometimes"
  JIRA helps assign stories and bugs to team members and track sprint completion.

Mini Case Study

Scenario: A software team keeps missing sprint targets.
Use of JIRA: The Scrum master tracks burndown chart daily, notices too many tasks stay "In Progress," and enforces smaller user stories plus daily review.
Outcome: Sprint predictability improves, and bug count reduces because testing is included inside the workflow.

Strengths	Limitations
• Excellent for Agile and software development • Strong bug tracking and reporting • Traceable change and release tracking	• Setup and workflow configuration takes time • Can feel complex for beginners • Requires discipline in updating status

Trello

What it is used for	Key features

Trello is a simple visual task management tool based on a **Kanban board**. It is popular for:
 	
• student projects
• small teams
• lightweight planning
	1. **Boards, lists, and cards**
	  o Board: Project
	  o Lists: To Do, Doing, Done
	  o Cards: Tasks
	2. **Checklists and due dates**
	Break a card into subtasks.
	3. **Labels and members**
	Labels for categories like UI, Backend, Testing.
	4. **Simple collaboration**
	Comments, attachments, notifications.

Example

Student project "Attendance System" board:

- To Do: Database design, Login UI
- Doing: QR scanner module
- Done: Requirement document

Mini Case Study

Scenario: A student group struggles to coordinate tasks.
Use of Trello: They create weekly task cards and assign members. Daily they move cards across lists.
Outcome: Everyone sees progress clearly, and tasks stop getting forgotten.

Strengths	Limitations
• Very easy to learn and use • Great for small teams and classroom projects • Visual and lightweight	• Limited advanced reporting • Not ideal for complex dependencies and baselines • Difficult to manage very large projects

Legal and Ethical Issues in Software Project Management

Software project management is not only about planning, coding, and delivery. Project managers are also responsible for ensuring that the project **follows the law and meets ethical standards**. Failure in these areas can lead to legal penalties, loss of reputation, and harm to users.

Legal Issues in Software Project Management

Meaning of Legal Issues

Legal issues refer to **laws, regulations, contracts, and rights** that must be followed during software development and delivery. A project manager must ensure compliance at every stage of the project life cycle.

1. Software Licensing Issues	2. Copyright and Intellectual Property Rights (IPR)
Explanation Software tools, libraries, and frameworks are protected by licenses. Using them without permission or violating license terms is illegal. **Common License Types** • Proprietary licenses • Open-source licenses • Freeware and shareware licenses **Example** Using a paid IDE or database without a valid license in a commercial project can lead to legal action. **Case Study** A startup used an open-source library that required source code disclosure but ignored the license terms. After release, they were forced to open their proprietary code, causing business loss.	**Explanation** Copyright protects original software code, documentation, designs, and content. Intellectual Property Rights define who owns the software. **Key Concerns** • Code plagiarism • Ownership disputes • Reuse of third-party code **Example** Copying code from the internet without permission or citation violates copyright laws. **Case Study** Two companies disputed ownership of a software product because the contract did not clearly define IP ownership. The case delayed product launch for months.

3. Contractual Obligations

Explanation

Software projects are governed by contracts that specify:

- Scope of work
- Payment terms
- Delivery deadlines
- Penalties for delay

Failure to meet contractual terms can result in legal disputes.

Example

If a project is delivered late without agreed extensions, the client may demand compensation.

4. Data Protection and Privacy Laws

Explanation

Software often handles personal and sensitive data. Laws require this data to be collected, stored, and processed securely.

Common Concerns

- Unauthorized data access
- Data leaks
- Improper data sharing

Example

Storing user passwords in plain text is a violation of data protection laws.

Case Study

A health application leaked patient data due to poor security, leading to legal penalties and loss of public trust.

5. Compliance with Industry Standards

Explanation

Certain domains require compliance with standards such as:

- Security standards
- Financial regulations
- Healthcare regulations

Example

Banking software must comply with security and audit requirements before deployment.

Ethical Issues in Software Project Management

Meaning of Ethical Issues

Ethical issues relate to **moral responsibility, fairness, honesty, and professional conduct**, even when no law is broken.

Ethics guide how decisions should be made to protect users, clients, and society.

1. Honesty and Transparency **Explanation** Project managers must report: • True project status • Realistic timelines • Actual risks **Example** Hiding delays to appear successful is unethical. **Case Study** A project manager falsely reported progress, leading to sudden project failure near deadline.	**2. Data Privacy and User Trust** **Explanation** Ethical responsibility goes beyond legal compliance. Users trust software with their data. **Example** Selling user data without consent, even if legal loopholes exist, is unethical.
3. Fair Treatment of Team Members **Explanation** Ethics require: • Fair workload distribution • Respectful behavior	**4. Avoiding Plagiarism** **Explanation** Using others' work without proper acknowledgment is unethical, even in academic projects. **Example**

• No discrimination **Example** Overburdening junior developers while seniors avoid responsibility is unethical.	Submitting copied project code as original work violates academic ethics.

5. Quality and Safety Responsibility

Explanation

Releasing knowingly defective software can harm users.

Example

Releasing a hospital system with known calculation errors risks patient safety.

Case Study

A software firm rushed a medical application to meet deadline, leading to incorrect dosage calculations. The ethical failure caused serious harm and legal consequences.

Role of Project Manager in Legal and Ethical Compliance

A project manager must:

- Understand relevant laws and licenses
- Ensure ethical decision-making
- Promote transparency
- Protect user interests
- Enforce professional standards

Relationship Between Legal and Ethical Issues

Legal	Ethical
Mandatory by law	Based on moral principles
Punishable if violated	Damages trust and reputation
Minimum requirement	Higher professional standard

Comprehensive Case Study: Educational Software Project

Problem	Impact	Ethical and Legal Resolution
A team used pirated software tools and copied code from online sources.	• Legal risk to institution • Academic misconduct • Loss of credibility	• Use licensed or open-source tools properly • Acknowledge sources • Train students on ethics

Advantages of Addressing Legal and Ethical Issues

- Avoid legal penalties
- Build user trust
- Improve software quality
- Enhance professional reputation